

1 Macro-Capitolo 1: Fondamenti, Obiettivi e Quadro Generale dei Sistemi Operativi

1.1 1.1 Definizione formale e collocazione sistemica

Un sistema operativo (SO) è un insieme strutturato di programmi (software) che svolge la duplice e cruciale funzione di gestire gli elementi fisici di un calcolatore (l'hardware) e di collocarsi come strato intermedio tra tale hardware e le applicazioni utente (p. 1). Questa posizione privilegiata lo rende il principale responsabile della mediazione tra le esigenze spesso conflittuali di utente e macchina. Da un lato, il SO deve facilitare l'uso del calcolatore, fornendo un'astrazione (un'interfaccia semplificata) che nasconda la complessità dei componenti fisici; dall'altro, deve sfruttare le risorse hardware nel modo più efficiente possibile. L'equilibrio tra questi due poli è estremamente delicato: un alto livello di astrazione rende l'uso più semplice ma spesso sacrifica le prestazioni, mentre un approccio meno astratto, che costringe l'utente a interagire direttamente con la macchina, aumenta l'efficienza a scapito della semplicità d'uso. In sintesi, il sistema operativo coordina l'intero ecosistema del calcolatore, gestendo sia le componenti hardware (dispositivi di I/O, dischi, memoria centrale) sia quelle software (file, variabili di programma, strutture dati) (p. 1).

1.2 1.2 Gli obiettivi strategici del sistema operativo

Gli obiettivi fondamentali che guidano la progettazione e il funzionamento di un SO sono riconducibili a due direttrici principali (p. 1). Il primo è la semplificazione dell'uso del sistema, che si traduce nell'offrire astrazioni di alto livello (come i file o i processi) che consentono a programmatori e utenti di interagire con la macchina senza dover conoscere i dettagli del sottostante hardware. Il secondo obiettivo, strettamente legato e antitetico al primo, è il raggiungimento della massima efficienza possibile: ogni astrazione introdotta ha un costo in termini di risorse (tempo di CPU, memoria) e il sistema deve quindi valutare il trade-off tra il livello di astrazione offerto e il suo impatto sulle prestazioni. Un esempio classico di questo contrasto è rappresentato dai diversi approcci dei sistemi operativi moderni: Windows, progettato con un'enfasi sull'interfaccia utente e l'astrazione, tende a essere meno efficiente a livello di risorse; al contrario, i sistemi Unix (e i loro derivati) sono storicamente meno astratti ma significativamente più efficienti in termini di utilizzo della CPU e della memoria (p. 1).

2 Macro-Capitolo 2: L'Evoluzione Storica dei Sistemi Operativi (Dalle Valvole al Time Sharing)

L'evoluzione dei sistemi operativi è un percorso affascinante che segue l'evoluzione tecnologica dell'hardware e che è stato costantemente animato da un principio ispiratore centrale: l'aumento dell'utilizzo del processore, riducendo i tempi di inattività della CPU (p. 2). Questa storia si articola classicamente in quattro generazioni distinte.

2.1 2.1 Prima Generazione (1946 – 1955): L'era dei calcolatori a valvole

Questa fase, immediatamente successiva alla Seconda Guerra Mondiale, vedeva calcolatori di dimensioni enormi basati su tecnologia a valvole termoioniche, impiegati prevalentemente per scopi militari (decifrazione di codici, calcolo di traiettorie balistiche) e governativi (p. 2). In questo contesto primordiale, il concetto di sistema operativo era del tutto assente. L'operatore coincideva con il programmatore, e l'accesso al calcolatore avveniva tramite un sistema di prenotazione. La

programmazione era un'operazione estremamente manuale e complessa: il programma veniva caricato in memoria un'istruzione alla volta, agendo fisicamente sugli interruttori posti sul pannello di controllo della console, e la sua esecuzione veniva monitorata tramite spie luminose che segnalavano eventuali errori (p. 2). Verso la fine di questa generazione si verificarono i primi, seppur limitati, miglioramenti: iniziarono a diffondersi periferiche come lettori e perforatori di schede, nastri magnetici e stampanti. Questi dispositivi richiedevano l'uso di programmi specifici (antenati dei moderni driver) per poter interagire con la CPU. Contestualmente, si iniziò a sviluppare il concetto di software di sistema, con la creazione delle prime librerie, compilatori, linker e loader. Nonostante questi progressi, le operazioni rimanevano estremamente complesse e l'efficienza complessiva del sistema era ancora drammaticamente bassa (p. 2).

2.2 Seconda Generazione (1955 – 1965): L'avvento dei Transistor e il Batch-ing

La seconda generazione segnò una svolta epocale con l'introduzione dei transistor, che sostituirono le ingombranti e inaffidabili valvole, rendendo i calcolatori più piccoli, veloci e affidabili. Questo salto tecnologico permise una prima, fondamentale separazione dei ruoli: l'operatore di macchina non era più necessariamente il programmatore, e i due potevano lavorare in parallelo, eliminando lo schema a prenotazione e riducendo i cosiddetti "tempi morti" (p. 2).

In questo contesto nacque l'idea rivoluzionaria del batching (raggruppamento). Il batching consisteva nell'organizzare i programmi (detti job) in sequenze omogenee per tipo di compilatore o linguaggio, al fine di minimizzare il tempo speso nei continui caricamenti e scaricamenti dei software di sistema (p. 3). Ad esempio, senza batching, una sequenza di job Fortran, Cobol, Fortran, Cobol avrebbe richiesto il caricamento del compilatore Fortran, poi Cobol, poi Fortran e infine Cobol con un enorme dispendio di tempo. Con il batching, i job venivano riordinati raggruppando tutti quelli in Fortran e poi tutti quelli in Cobol, consentendo di caricare il compilatore Fortran una sola volta e, analogamente, quello Cobol una sola volta, con un notevole risparmio di tempo (p. 3).

Per gestire il passaggio da un job all'altro in modo automatico, fu necessario introdurre un software sempre attivo in memoria, chiamato monitor residente. Questo monitor era l'antesignano del kernel moderno e si componeva di tre parti fondamentali: i driver per interagire con l'hardware, un sequenziatore di job (antenato dello scheduler) che determinava l'ordine di esecuzione dei programmi, e un interprete che permetteva la comunicazione con l'operatore tramite schede di controllo (p. 3). La memoria, di conseguenza, iniziò a essere suddivisa in una parte per i programmi utente e una parte riservata al monitor residente (p. 3).

2.2.1 Limiti del batch tradizionale e la soluzione off-line

Nonostante i progressi, persistevano problemi critici: l'utilizzo della CPU era ancora scarsamente efficiente a causa della velocità del I/O, nettamente inferiore a quella della CPU (p. 4). Un esempio illuminante riportato dal testo (p. 4) mostra come un compilatore potesse processare 300 schede al secondo, mentre il lettore di schede ne leggeva solo 20 al secondo. Per un programma di 1200 schede, la CPU impiegava solo 4 secondi per l'elaborazione, ma il sistema era bloccato per 60 secondi nelle operazioni di I/O, portando a un utilizzo della CPU di appena il 6.25%. La soluzione a questa strozzatura fu il passaggio dal batch tradizionale alle operazioni off-line (p. 4). L'idea chiave era quella di sovrapporre le operazioni di I/O all'utilizzo della CPU. Invece di avere il flusso diretto "lettore di schede → CPU → stampante", si inserivano dei nastri magnetici come buffer intermedi. Il flusso off-line prevedeva che un lettore di schede trasferisse i dati su un nastro magnetico (nastro

out/in). Successivamente, questo nastro veniva collegato alla CPU, che lo leggeva, elaborava i dati e scriveva l'output su un secondo nastro. Infine, questo secondo nastro veniva trasferito su una stampante per la produzione dell'output fisico. Questo schema, specialmente con più lettori di schede, permetteva di liberare la CPU da lunghi tempi di attesa, facendola lavorare in parallelo rispetto alle operazioni di lettura e stampa (p. 4).

2.2.2 2.2.2 Sovrapposizione su singola macchina: Polling, Interrupt e DMA

L'evoluzione successiva fu quella di realizzare la sovrapposizione di CPU e I/O all'interno di una singola macchina, senza bisogno di macchine indipendenti (p. 5). Questo divenne possibile grazie a due meccanismi architetturali fondamentali. Il primo era il meccanismo di interazione tradizionale, il polling (interrogazione a polling), in cui la CPU, tramite istruzioni bloccanti, interrompeva il proprio lavoro per chiedere continuamente alla periferica se il trasferimento dati fosse concluso; se non lo era, la CPU rimaneva in attesa attiva, sprecando cicli (p. 5). Il secondo, molto più evoluto, era il meccanismo asincrono, basato su Interrupt e DMA (Direct Memory Access). Con questo approccio, la richiesta di I/O non è più bloccante: la periferica stessa avvisa la CPU tramite un segnale di interrupt solo quando ha completato il proprio compito, lasciando la CPU libera di eseguire altre elaborazioni (p. 5). La CPU, tramite il device driver, fornisce le istruzioni alla periferica, che si dedica al compito in modo autonomo. Per evitare un'eccessiva frammentazione del lavoro della CPU dovuta a un flusso troppo fitto di interrupt, entra in gioco il DMA. Il DMA (p. 5) è un controllore hardware specializzato che si occupa del trasferimento di interi blocchi di dati tra la periferica e la memoria senza l'intervento diretto della CPU, inviando un singolo interrupt per blocco di dati completato, anziché per ogni singolo byte, incrementando drasticamente l'efficienza.

2.2.3 2.2.3 Buffering e Spooling: Dall'I/O sequenziale al pool di job

Grazie agli interrupt e al DMA, divenne possibile sovrapporre la CPU e le operazioni di I/O relative allo stesso job, dando vita al concetto di buffering (p. 6). Mentre la CPU elabora dati precedentemente caricati, le periferiche di I/O preparano i dati successivi, mantenendo il flusso attivo fino a quando la velocità della CPU e quella del I/O sono bilanciate. L'evoluzione successiva fu il simultaneous peripheral operations on-line (spooling) (p. 6). Con lo spooling, si passa dai lenti nastri magnetici sequenziali ai più veloci dischi magnetici ad accesso casuale. L'accesso casuale consente di non dover più riavvolgere i nastri e si introduce il concetto di pool di job: un'area su disco dove risiedono gruppi di job pronti per essere eseguiti. I lettori di schede trasferiscono i dati su questo disco, che comunica on-line con la CPU. Il disco agisce come un enorme buffer, e il sistema operativo, tramite lo job scheduling, ha il compito di gestire cosa viene memorizzato, la disponibilità dei dati e il tempo da assegnare a ogni programma. Questo meccanismo abbandona il paradigma precedente in cui i dati dovevano essere letti sequenzialmente dal singolo job, gettando le basi per i moderni sistemi con programmi caricati in memoria (p. 7).

2.3 2.3 Terza Generazione (1965 – 1980): Multiprogrammazione, Time Sharing e Protezione

L'avvento dei circuiti integrati portò a un'ulteriore accelerazione e alla nascita della multiprogrammazione (p. 7). Mentre nell'esecuzione sequenziale i job vengono eseguiti uno dopo l'altro, nella multiprogrammazione l'elaborazione di un job da parte della CPU può avvenire in contemporanea con le operazioni di I/O di un altro, mantenendo la CPU sempre occupata. Mentre i sistemi batch tradizionali puntavano a minimizzare il tempo di completamento, i sistemi multiprogrammati erano votati a soddisfare utenti interattivi, rendendo fondamentale il tempo di risposta (p.

7). Un'estensione logica della multiprogrammazione è il time sharing (o multitasking), in cui la CPU dedica intervalli di tempo specifici all'esecuzione di diversi job in rapida rotazione. Questo dà all'utente l'illusione che tutti i programmi funzionino in parallelo, mentre in realtà il tempo di CPU viene suddiviso a rotazione tra i vari processi secondo politiche di scheduling definite. È cruciale sottolineare che multiprogrammazione e time sharing non sono sinonimi: la multiprogrammazione mira a tenere occupata la CPU, mentre il time sharing mira a fornire un'interfaccia interattiva e reattiva a più utenti (p. 7). Con il time sharing nascono i sistemi "moderni" dotati di periferiche interattive (tastiera, monitor) e l'astrazione del file system per un accesso logico ai dati (p. 8).

2.3.1 2.3.1 La nascita dei meccanismi di protezione

La condivisione delle risorse introdusse il problema critico della protezione (p. 8). Nei sistemi originari, non essendoci concorrenza, un processo non rischiava di scrivere nell'area di memoria di un altro. Con la multiprogrammazione, divenne essenziale prevenire interferenze. La protezione si articola su tre fronti:

- **Protezione dell'I/O:** si passa da un SO eseguito come programma qualunque a un SO eseguito in dual mode. Vengono introdotte la modalità utente (USER), in cui i job non possono accedere direttamente alle risorse I/O, e la modalità supervisore (KERNEL), in cui il sistema operativo ha pieno accesso. Le istruzioni di I/O sono privilegiate e vengono invocate tramite system call, che generano interrupt software per passare dalla modalità utente a quella kernel. Al termine, il SO ripristina la modalità utente (p. 8).
- **Protezione della memoria:** il SO assegna a ogni processo specifiche aree di memoria, annotando le zone assegnate, per impedire che un processo scriva in aree non di sua competenza (p. 8).
- **Protezione della CPU:** il SO mantiene il controllo attraverso algoritmi di scheduling e l'uso di timer. Se un processo non restituisce il controllo entro un tempo prestabilito, viene automaticamente interrotto per garantire che nessun programma possa monopolizzare la CPU (p. 8).

2.4 2.4 Quarta Generazione (1980 – oggi): Diversificazione Architetture

Dagli anni '80 in poi, le innovazioni radicali sul fronte dei sistemi operativi si sono attenuate, lasciando spazio a una forte diversificazione delle architetture per adattarsi ai diversi contesti d'uso:

- **SO per PC e workstation:** progettati per l'utilizzo personale e interattivo.
- **SO di rete:** offrono risorse remote, ma l'accesso a queste risorse è logicamente separato e diverso dall'accesso a quelle locali.
- **SO distribuiti:** in cui non esiste separazione logica; l'accesso a una risorsa remota è trasparente e identico a quello di una risorsa locale.
- **SO real-time:** caratterizzati da vincoli temporali stringenti (es. sistemi di controllo del traffico aereo).
- **SO embedded:** progettati per sistemi dedicati e applicazioni specifiche (p. 8).

3 Macro-Capitolo 3: Architettura e Componenti Fondamentali del Sistema Operativo

3.1 3.1 Componenti chiave e loro responsabilità

Un sistema operativo moderno è un'entità complessa composta da diversi moduli interconnessi, ognuno con una responsabilità ben definita (p. 9-10):

- **Gestione dei processi:** è il cuore del SO. Si occupa di creare e terminare i processi, sospenderli e riprenderli, sincronizzarli e permettere la loro comunicazione.
- **Gestione della memoria primaria (RAM):** conserva i dati condivisi tra CPU e I/O durante l'esecuzione dei programmi. Le sue responsabilità includono la gestione dello spazio di archiviazione, la conoscenza delle zone di memoria occupate, la decisione su quali processi caricare in memoria e la gestione dell'allocazione e del rilascio degli spazi di memoria.
- **Gestione della memoria secondaria (dischi):** poiché la RAM è volatile e limitata, la memoria secondaria è indispensabile per l'archiviazione permanente dei dati. Il SO gestisce lo spazio libero su disco, alloca lo spazio per i file e ordina gli accessi ai dispositivi di memorizzazione.
- **Gestione dell'I/O:** ha il compito di fare astrazione dei dispositivi fisici, nascondendone le specificità all'utente. Gestisce un'interfaccia generica verso i device driver e un sistema di buffering per accumulare gli accessi.
- **Gestione dei file:** fornisce un'astrazione logica dei dati (i file) indipendentemente dal supporto fisico. Il SO mette a disposizione un file system per creare e cancellare file e directory, primitive per la loro gestione, la corrispondenza tra spazio logico e fisico e meccanismi di backup.
- **Protezione:** controlla l'accesso alle risorse da parte di utenti e processi, definendo accessi autorizzati e fornendo strumenti per verificare le politiche di accesso.
- **Rete (sistemi distribuiti):** in questi sistemi, il SO è responsabile della gestione trasparente di processi, memoria e file system distribuiti (p. 11).

3.2 3.2 L'interprete dei comandi (Shell) e le System Call

L'interfaccia utente primaria (storicamente) è l'interprete dei comandi (shell) (p. 12). La shell è un programma che legge e interpreta le istruzioni inserite dall'utente, consentendo di creare e gestire processi, gestire I/O, dischi, memoria, file system, protezioni e rete (p. 12). La comunicazione tra i programmi in esecuzione e il sistema operativo avviene tramite le system call. La proporzione è: utente sta a shell come programma sta a system call (p. 12). Le system call (p. 12) sono funzioni con parametri di input e valori di ritorno che i programmi utilizzano per richiedere servizi al kernel (es. leggere un file, allocare memoria). Il passaggio dei parametri può avvenire in diversi modi: tramite registri della CPU, tramite lo stack del programma, o tramite una tabella in memoria il cui indirizzo viene passato al SO (p. 12). Il flusso di una system call è altamente strutturato (p. 13):

1. Il programma chiama una funzione di libreria $A(x)$.
2. Il parametro x viene memorizzato nello stack (o in una tabella).

3. La funzione di libreria invoca la vera system call `_A`, che deposita il numero identificativo della chiamata in un punto noto al SO.
4. Viene eseguita l'istruzione TRAP, che è un interrupt software, provocando il passaggio dalla modalità utente alla modalità kernel.
5. La CPU salta a un indirizzo fisico predefinito, ovvero il gestore degli interrupt.
6. Il SO, leggendo il numero della system call, smista la chiamata al gestore specifico.
7. Il gestore esegue la routine richiesta.
8. Al termine, il controllo ritorna alla funzione di libreria, che restituisce il risultato al programma chiamante.

Un'architettura alternativa, utilizzata per gestire un numero elevato di parametri, prevede il caricamento dei parametri in un'area di memoria definita (tabella) e il passaggio del puntatore a quest'area tramite un registro, anziché utilizzare lo stack (p. 13).

3.3 Programmi di sistema

Oltre al kernel e alla shell, il sistema operativo include una suite di programmi di sistema (p. 14). Questi sono applicazioni che, pur non facendo parte integrante del kernel, forniscono servizi essenziali e sono utilizzabili anche dall'utente per operazioni come la gestione e manipolazione dei file (creare, copiare, cancellare), la richiesta di informazioni sullo stato del sistema (data, memoria disponibile), strumenti di supporto alla programmazione (compilatori, assembleri), formattazione di documenti, programmi di gestione della rete e altre utility. È fondamentale comprendere che questi programmi servono per gestire il SO ma non sono parte integrante del kernel (p. 14).

4 Macro-Capitolo 4: Strutture Architetture dei Sistemi Operativi

La progettazione dell'architettura del sistema operativo è cruciale per le sue prestazioni, manutenibilità e affidabilità. Il documento esamina diverse strutture, ognuna con i propri pregi e difetti.

4.1 Sistemi Monolitici

I sistemi monolitici rappresentano la struttura più primitiva (p. 14). In questa architettura, non esiste una gerarchia tra software e hardware, ma un unico grande strato di sistema operativo che si frappone tra l'utente e la macchina. Tutte le funzionalità sono implementate direttamente e risiedono allo stesso livello. Il vantaggio principale di questo approccio è la velocità di scrittura del codice e l'efficienza, poiché non ci sono livelli di astrazione aggiuntivi. Tuttavia, la mancanza di modularità e di confini netti tra le componenti rende il sistema estremamente complesso da gestire quando cresce di dimensioni. L'aggiunta di una nuova funzionalità o la modifica di quelle esistenti richiede di ritoccare l'intero codice, e le modifiche a livello hardware si propagano in tutto il sistema, rendendo difficile la manutenzione e l'estensione (p. 14).

4.2 Sistemi a Struttura Semplice

Per ovviare ai problemi dei sistemi monolitici, si è introdotta un'organizzazione gerarchica in livelli separati, ognuno con compiti specifici, dando vita ai sistemi a struttura semplice (p. 15). Esempi emblematici sono l'originale MS-DOS e il primo UNIX.

- **MS-DOS:** aveva una struttura a livelli in cui quello più basso era il ROM BIOS device driver e quello più alto i programmi applicativi. I livelli superiori potevano interfacciarsi con quelli inferiori, ma non viceversa. Questo sistema, pensato per fornire molte funzionalità in poco spazio, manteneva una struttura minima e non prevedeva il dual mode, poiché l'hardware (Intel 8088) non lo supportava (p. 15).
- **UNIX (originale):** separava nettamente i programmi di sistema, eseguiti in modalità utente, dal kernel, il nucleo centrale eseguito in modalità kernel. Il kernel racchiudeva tutto ciò che stava tra le system call e l'hardware sottostante (p. 15).

4.3 Sistemi a Livelli

Il sistema a livelli rappresenta un'estensione più formale della struttura semplice, prevedendo una pila di livelli in cui ogni strato *i*-esimo comunica solo con lo strato immediatamente superiore e quello immediatamente inferiore (p. 16). Questo approccio massimizza la modularità, facilitando l'ottimizzazione, la manutenzione e il debugging. Tuttavia, l'introduzione di numerosi livelli aumenta i ritardi di comunicazione, riducendo l'efficienza, e diminuisce la portabilità, poiché le funzionalità dipendenti dall'architettura possono essere sparse su più livelli. L'esempio storico per eccellenza è il sistema operativo THE, sviluppato da Dijkstra nel 1968, che prevedeva cinque livelli: dal livello 0 (hardware) fino al livello 5 (programmi utente), passando per la gestione della CPU (livello 1), della memoria (livello 2), la console (livello 3) e l'I/O (livello 4) (p. 16).

4.4 Virtual Machine

La virtual machine (VM) estremizza l'approccio a livelli, astruendo non solo il software ma l'intero hardware (p. 17). Crea una separazione logica dell'hardware, dando a ogni utente l'illusione di avere una macchina fisica dedicata. I vantaggi sono notevoli: protezione completa (ogni VM è isolata), possibilità di eseguire più SO diversi sulla stessa macchina host, ottimizzazione delle risorse e un ambiente ideale per lo sviluppo di nuovi SO in contemporanea con il sistema standard (p. 17). Gli svantaggi includono il degrado delle prestazioni dovuto alla condivisione delle risorse, la necessità di gestire un dual mode virtuale e l'isolamento delle VM, che per comunicare necessitano di meccanismi software come condivisione di volumi o reti virtuali. Oggi le VM sono ancora ampiamente utilizzate in contesti come l'esecuzione di sistemi legacy (es. MS-DOS su Windows), piattaforme di virtualizzazione come VMware e VirtualBox, la Java Virtual Machine (JVM) e i container Linux come Docker (p. 17).

4.5 Sistemi Client-Server (Micro-kernel)

Nei sistemi client-server, il kernel si riduce al minimo (micro-kernel), occupandosi solo della comunicazione tra processi (p. 18). Tutti gli altri servizi del SO (come il file system, il gestore della memoria, il gestore I/O) sono realizzati come processi utente (server). Un processo client richiede un servizio inviando un messaggio a un server, che dopo l'esecuzione restituisce il risultato. Questo modello, adatto soprattutto ai sistemi operativi distribuiti, offre una grande flessibilità e robustezza, ma può introdurre overhead di comunicazione (p. 18).

5 Macro-Capitolo 5: Il Concetto di Processo e la sua Rappresentazione in Memoria

5.1 5.1 Definizione formale di processo e differenza dal programma

Uno dei concetti più fondamentali nei sistemi operativi è la distinzione tra programma e processo (p. 19). Un programma è un'entità statica, un insieme di istruzioni e dati memorizzato in uno o più file su disco; rimane immutato finché non viene eseguito. Un processo, invece, è un'entità dinamica: è un'istanza di un programma in esecuzione. È il programma in attività, che utilizza risorse di sistema come la memoria RAM, il tempo di CPU, le periferiche e i meccanismi di protezione. Uno stesso programma, avviato più volte, genera più processi indipendenti, ognuno con la propria area dati e il proprio stato di esecuzione, che possono produrre risultati diversi (p. 19).

5.2 5.2 L'immagine in memoria del processo (Layout di memoria)

Per essere eseguito, un programma deve essere caricato in memoria, creando quella che viene definita immagine in memoria del processo (p. 19). Questa immagine è suddivisa in quattro sezioni logiche (p. 19-20):

- **Codice (Text):** contiene le istruzioni eseguibili del programma. È una sezione statica, che non muta durante l'esecuzione.
- **Dati (Data):** contiene le variabili globali del programma (statiche e globali). Anche questa sezione ha dimensioni fisse.
- **Stack:** è una memoria temporanea utilizzata per le chiamate di funzione. Ospita i parametri passati alle funzioni, gli indirizzi di ritorno e le variabili automatiche locali. Lo stack cresce e si riduce dinamicamente durante l'esecuzione.
- **Heap:** è l'area di memoria allocata dinamicamente a runtime tramite chiamate di sistema come malloc e free. Anche l'heap cresce e decresce dinamicamente (p. 20).

5.3 5.3 Il Process Control Block (PCB) - Attributi del Processo

Ogni processo è rappresentato all'interno del sistema operativo da una struttura dati critica chiamata Process Control Block (PCB), o Task Control Block (TCB) (p. 21). Il PCB è il deposito di tutte le informazioni necessarie per gestire e controllare il processo. I suoi campi principali sono:

- **Stato del processo:** indica se il processo è in esecuzione, in attesa, pronto, ecc.
- **Program counter:** contiene l'indirizzo della prossima istruzione da eseguire.
- **Registri della CPU:** salvano il contenuto di tutti i registri del processore, fondamentali per poter riprendere l'esecuzione dopo un'interruzione.
- **Limiti di memoria (Memory limits):** informazioni per delimitare l'area di memoria assegnata al processo, prevenendo accessi a zone di altri processi.
- **Informazioni sullo scheduling della CPU:** priorità del processo, puntatori alle code di scheduling.
- **Informazioni sulla gestione della memoria:** registri di base e limite, tabelle delle pagine o dei segmenti.

- **Informazioni di accounting:** tempo di CPU utilizzato, limiti di tempo, ID del processo.
- **Informazioni sullo stato dell'I/O:** lista dei dispositivi di I/O assegnati, file aperti (p. 21).

6 Macro-Capitolo 6: Stati di un Processo e Meccanismi di Scheduling

6.1 6.1 Gli stati fondamentali e le loro transizioni

Un processo non è mai in uno stato statico, ma subisce una continua evoluzione durante il suo ciclo di vita, transitando tra diversi stati (p. 22):

- **Nuovo (New):** il processo è in fase di creazione, il SO sta allestendo il PCB e l'immagine in memoria.
- **Pronto (Ready):** il processo è stato creato e caricato in memoria, ha tutte le risorse necessarie tranne la CPU, ed è in attesa di essere assegnato a un core elaborativo. Si trova nella coda dei processi pronti.
- **In esecuzione (Running):** il processo è attualmente in esecuzione sulla CPU, e le sue istruzioni vengono effettivamente eseguite. In un sistema single-core, solo un processo può essere in questo stato per volta.
- **In attesa (Waiting):** il processo è sospeso e non può essere eseguito perché sta attendendo il verificarsi di un evento esterno, come il completamento di un'operazione di I/O o la ricezione di un segnale.
- **Terminato (Terminated):** il processo ha completato la sua esecuzione e sta per essere rimosso dal sistema, liberando le risorse allocate.

Le transizioni tra questi stati sono gestite dal sistema operativo (p. 23). Quando un programma viene avviato, il SO crea il processo (stato Nuovo) e lo ammette nel sistema, portandolo allo stato Pronto. Lo scheduler, tramite l'operazione di dispatch, assegna la CPU al processo, che passa in Esecuzione. Il processo può poi essere interrotto da un time-out (quantum di tempo scaduto), tornando in coda dei Pronti, oppure andare in Attesa per un evento di I/O. Quando l'evento si verifica, il processo torna in Pronto. Questo ciclo prosegue fino al completamento, quando il processo termina. È importante notare che i nomi degli stati possono variare da SO a SO, ma il modello concettuale è universale (p. 23).

6.2 6.2 Schedulazione e code di scheduling

L'obiettivo della multiprogrammazione è tenere la CPU sempre occupata; quello del time-sharing è fornire tempi di risposta rapidi. Lo scheduler dei processi seleziona il processo da eseguire (p. 23). I processi sono gestiti tramite code:

- **Coda dei processi pronti (Ready Queue):** una lista concatenata di PCB di tutti i processi in stato Pronto, pronti per essere eseguiti.
- **Code di attesa (Wait Queues):** diverse code per ciascun evento o risorsa (es. una coda per la stampante, una per il disco). I processi che hanno richiesto una risorsa non ancora disponibile vengono spostati nella coda di attesa corrispondente (p. 23).

I processi vengono anche classificati in base al loro comportamento: I/O-bound, che passano la maggior parte del tempo in operazioni di I/O, e CPU-bound, che passano la maggior parte del tempo in elaborazione (p. 23). Il diagramma di accodamento (p. 24) illustra il flusso: un processo inizia nella coda dei pronti, viene selezionato per l'esecuzione, e a seconda che necessiti di I/O, crei un figlio, o termini il quanto di tempo, viene spostato nelle relative code di attesa o nuovamente in quella dei pronti. Lo swapping (p. 25) è una forma intermedia di scheduling che permette di rimuovere temporaneamente un processo dalla memoria (swapped out) verso il disco, per liberare spazio, e successivamente ricaricarlo (swapped in) per riprendere l'esecuzione.

6.3 Il Dispatcher e il Context Switch

L'operazione di dispatch è eseguita dal dispatcher, una componente del SO che si occupa del cambio di contesto, del salvataggio e caricamento del PCB, e della gestione della modalità di esecuzione (p. 25). Il cuore di questa operazione è il context switch (p. 25). Ogni volta che la CPU deve passare da un processo a un altro, il sistema salva lo stato del contesto (valori dei registri, PC, stato) del processo in uscita nel suo PCB, e carica lo stato del processo in entrata. Questo è un passaggio oneroso in termini di tempo (qualche millisecondo) e dipende dalla velocità della memoria, dal numero di registri e dalla complessità del SO (p. 26).

7 Macro-Capitolo 7: Operazioni su Processi (Creazione e Terminazione)

7.1 Creazione di un processo (fork ed exec)

Nei sistemi moderni, i processi sono generati dinamicamente. Un processo può creare nuovi processi, chiamati processi figli. Il processo creante è detto processo genitore (o padre). Questa relazione gerarchica forma un albero di processi, identificati da un numero univoco, il PID (process identifier) (p. 26). In Linux, systemd (PID 1) è il processo padre di tutti i processi utente (p. 27). La creazione di un figlio avviene principalmente attraverso la chiamata di sistema `fork()` (p. 27). `fork()` duplica il processo chiamante: il figlio è una copia quasi perfetta del padre, ottenendo una copia dei segmenti di testo, dati e stack. Se `fork()` ha successo, restituisce il PID del figlio nel processo padre e 0 nel processo figlio; in caso di errore restituisce -1. Subito dopo la `fork()`, i due processi eseguono lo stesso codice indipendentemente. Per far sì che il figlio esegua un programma diverso, si utilizza una funzione della famiglia `exec()`. La `exec()` carica in memoria un nuovo file binario, sovrascrivendo completamente l'immagine di memoria del processo corrente (codice, dati, stack, heap), e ne avvia l'esecuzione (p. 27). Se `exec()` ha successo, non ritorna mai al chiamante; in caso di errore, restituisce -1. Questo meccanismo è alla base del funzionamento delle shell, che usano `fork()` per creare un figlio e `exec()` per eseguire un comando (es. `/bin/ls`). Il processo padre, dopo aver creato il figlio, può attendere la sua terminazione con la chiamata `wait()` (p. 27). L'esempio codice a p. 28 mostra chiaramente questo flusso.

7.2 Terminazione di un processo

Un processo termina normalmente quando esegue l'ultima istruzione e chiama la system call `exit()`, che comunica al SO di cancellare il processo (p. 29). Il processo figlio restituisce un'informazione di stato al genitore, che la riceve tramite `wait()`. La terminazione può anche essere forzata dal processo genitore (es. `TerminateProcess` in Windows), o verificarsi a causa di errori (divisione per zero) o per superamento delle risorse assegnate (p. 29). Se un processo termina, i suoi figli possono

essere terminati a cascata, a seconda delle politiche del SO. Un concetto critico è quello del processo zombie (p. 29): si verifica quando un processo termina, ma il suo processo genitore non ha ancora invocato `wait()` per raccogliergli lo stato di uscita. Il processo rimane nella tabella dei processi come zombie, occupando solo la voce del PCB, fino a quando il genitore non chiama `wait()`. Al contrario, se un processo genitore termina prima dei suoi figli, questi diventano processi orfani. In sistemi Linux/Unix, i processi orfani vengono adottati da `systemd` (o `init`), che periodicamente invoca `wait()` per liberarne le risorse (p. 30).

8 Macro-Capitolo 8: Comunicazione tra Processi (IPC) e Modelli di Kernel

8.1 8.1 Processi Indipendenti e Cooperanti

I processi in un sistema possono essere classificati in due categorie in base alla loro interazione (p. 30):

- **Processi indipendenti:** non possono influire sugli altri processi né esserne influenzati. Non condividono dati e la loro esecuzione è deterministica, dipendendo solo dai propri input.
- **Processi cooperanti:** possono influire sugli altri processi o esserne influenzati. Condividono dati e informazioni, e la loro esecuzione è nondeterministica, poiché dipende dalla velocità relativa di esecuzione degli altri processi. La cooperazione è utile per la condivisione di informazioni, la velocizzazione del calcolo (parallelizzazione) e la modularità del sistema (p. 30).

8.2 8.2 Modelli di Comunicazione tra Processi (IPC)

Per consentire la comunicazione, i processi cooperanti necessitano di meccanismi di InterProcess Communication (IPC). I due modelli fondamentali sono (p. 31):

- **Scambio di messaggi (Message Passing):** la comunicazione avviene tramite lo scambio esplicito di messaggi tra processi. Utile per piccole quantità di dati e relativamente semplice da implementare, ma comporta l'overhead di continue chiamate di sistema al kernel.
- **Memoria condivisa (Shared Memory):** viene creata una regione di memoria che può essere acceduta da più processi. La comunicazione avviene direttamente tramite letture e scritture in quest'area. Questo modello è più veloce dello scambio di messaggi perché, una volta che la memoria è condivisa, non richiede più l'intervento del kernel; richiede però l'uso di meccanismi di sincronizzazione per evitare conflitti di accesso (p. 31).

8.3 8.3 Modelli di Esecuzione del Kernel

Il documento conclude esaminando come il codice del sistema operativo (il kernel) viene eseguito rispetto ai processi utente, delineando tre possibili architetture (p. 31-33):

- **Kernel separato:** il kernel esiste al di fuori di ogni processo utente, in uno spazio di memoria riservato. La sua esecuzione è indipendente e in modalità kernel. È tipico dei primi SO.
- **Kernel eseguito all'interno dei processi utente (Processo come contenitore):** in questo modello, il codice del SO è incluso nello spazio di indirizzamento di ogni processo utente (p. 32). L'immagine in memoria del processo include, oltre a codice, dati, stack e heap, anche un kernel stack e uno spazio di indirizzamento condiviso contenente il codice del SO.

La grande efficienza di questo modello risiede nel fatto che una chiamata al SO non richiede un costoso context switch tra processi, ma solo un più rapido mode switch (passaggio da modalità utente a modalità kernel e viceversa). Quando un processo invoca una system call, il controllo passa semplicemente al codice del kernel all'interno del suo stesso spazio di indirizzi. Al termine, il kernel può decidere se restituire il controllo al processo chiamante o passare a un altro processo.

- **Kernel come processo a sé stante:** in questa terza opzione, il SO viene gestito come un insieme di processi analoghi a quelli utente, eseguiti in modalità kernel. Ogni servizio del SO ha il suo processo individuale. Questo modello, pur perdendo il vantaggio del puro mode switch (richiedendo un context switch per ogni chiamata), si adatta meglio alle architetture multiprocessore, dove i processi del sistema possono essere eseguiti in parallelo su core diversi rispetto ai processi utente, migliorando la scalabilità (p. 33).

9 Macro-Capitolo 1 (Thread): Il Modello a Thread e la Programmazione Concorrente

9.1 1.1 Definizione Formale di Thread e Distinzione dal Processo

Il concetto di thread rappresenta un'evoluzione del modello di processo classico. In un sistema operativo moderno, un processo non è più un'entità monolitica, ma un contenitore di risorse. La thread (p. 1) è definita come l'unità base di utilizzo della CPU, ovvero il percorso di controllo (flusso di esecuzione) all'interno di un processo. In dettaglio, una thread è composta da un identificatore (TID), un program counter, un insieme di registri e una pila (stack) dedicata, che conserva la cronologia delle chiamate e le variabili locali.

La distinzione fondamentale è che il processo possiede le risorse (spazio di indirizzamento, file aperti, ecc.), mentre la thread esegue il codice. Di conseguenza, un processo tradizionale, detto processo pesante, contiene una singola thread, mentre un processo multithread (p. 1) ne contiene multiple, le quali condividono la sezione di codice, i dati globali e le risorse di sistema del processo padre. Questa separazione logica tra "possesso" (processo) ed "esecuzione" (thread) è il paradigma alla base della concorrenza moderna.

9.2 1.2 Vantaggi della Programmazione Multithread (p. 2)

L'adozione del multithreading comporta quattro categorie di vantaggi sostanziali:

- **Tempo di risposta (Responsiveness):** In un'applicazione interattiva, il multithreading consente al programma di rimanere reattivo all'utente anche se una parte di esso è bloccata in un'operazione lunga. Ad esempio, se il thread principale dell'interfaccia utente delega una scrittura su disco a un thread separato, l'interfaccia non si "congela", migliorando l'esperienza d'uso.
- **Condivisione delle risorse (Resource Sharing):** I processi, per scambiarsi dati, devono ricorrere a meccanismi complessi come la memoria condivisa o lo scambio di messaggi. I thread, invece, condividono per default l'intero spazio di memoria e le risorse del processo proprietario, semplificando la comunicazione.
- **Economia (Economy):** Creare un thread è un'operazione molto meno onerosa in termini di tempo e memoria rispetto alla creazione di un nuovo processo. Allocare un nuovo spazio

di indirizzi e una nuova tabella dei file è costoso; la creazione di un thread all'interno di un processo esistente richiede risorse minime. Anche il cambio di contesto (context switch) tra thread dello stesso processo è più leggero, poiché non richiede il ricaricamento della Translation Lookaside Buffer (TLB) o il cambio dello spazio di indirizzamento.

- **Scalabilità (Scalability):** In architetture multicore (p. 3), il multithreading consente il vero parallelismo. Thread diversi possono essere eseguiti simultaneamente su core di elaborazione distinti. Un processo a singolo thread, al contrario, può utilizzare al massimo un singolo core, indipendentemente da quanti siano disponibili.

9.3 1.3 Implementazioni dei Thread: Livello Utente e Livello Kernel (p. 3)

Esistono due modalità principali di gestione dei thread, a seconda del supporto fornito dal sistema operativo:

- **Thread a livello utente (User-Level Threads - ULT):** Sono gestiti interamente da una libreria a livello utente, senza che il kernel ne sia a conoscenza. Il kernel vede il processo come un'entità unica e monolitica. Lo scheduling dei thread è gestito dalla libreria. Problema critico (p. 4): se una thread esegue una chiamata di sistema che la blocca (es. I/O), l'intero processo viene bloccato, perché il kernel non ha visibilità delle altre thread interne. Inoltre, in un sistema single-core, il kernel assegna la CPU al processo, non alla singola thread, impedendo lo sfruttamento del parallelismo su più core.
- **Thread a livello kernel (Kernel-Level Threads - KLT):** Sono gestiti direttamente dal kernel del sistema operativo. Il SO mantiene una struttura di controllo per ogni thread (Thread Control Block - TCB). In questo caso, se una thread si blocca, il kernel può schedare un'altra thread dello stesso processo, garantendo la continuità. Questa è la soluzione adottata dalla quasi totalità dei sistemi operativi moderni (Windows, Linux, macOS). Un esempio pratico visibile in Linux è il thread kthreadd (PID 2), che funge da genitore per tutti i thread a livello kernel (p. 2).

Esistono tre modelli di relazione tra thread utente e thread kernel (p. 4-5):

- **Modello Many-to-One:** Molte thread utente sono mappate su un singolo thread kernel. (Obsoleto, soffre del problema del blocco).
- **Modello One-to-One:** Ogni thread utente è mappato su un thread kernel. Offre vero parallelismo ma ha overhead di creazione (usato da Linux, Windows).
- **Modello Many-to-Many:** Molte thread utente sono multiplexate su un numero minore o uguale di thread kernel. Combina flessibilità ed economia, permettendo al sistema di bilanciare il carico.

10 Macro-Capitolo 2 (Thread): Fondamenti dello Scheduling della CPU

Lo scheduling della CPU (p. 7) è il meccanismo fondamentale che consente la multiprogrammazione. Il suo obiettivo è mantenere la CPU occupata il più possibile, passando rapidamente da un processo all'altro.

10.1 2.1 Il Ciclo CPU-I/O (CPU-I/O Burst Cycle) (p. 7)

L'esecuzione di un processo consiste in un'alternanza di due fasi:

- **CPU Burst:** sequenza di operazioni di calcolo svolte dalla CPU.
- **I/O Burst:** sequenza di operazioni di attesa per il completamento di un'operazione di input/output.

La distribuzione della durata dei CPU burst è tipicamente caratterizzata da molte brevi sequenze e poche sequenze molto lunghe. Questa curva di distribuzione è cruciale per la progettazione degli algoritmi di scheduling. Un processo I/O-bound presenta molti burst brevi, mentre un processo CPU-bound presenta burst lunghi e poco frequenti.

10.2 2.2 Le Tipologie di Scheduler (p. 9)

Il sistema operativo utilizza tre tipi di scheduler:

- **Scheduler a lungo termine (Long-Term Scheduler):** Seleziona i processi dal pool dei job in memoria secondaria e li carica in memoria per l'esecuzione. Nei sistemi moderni, questo scheduler è spesso assente o molto semplificato, poiché la memoria è abbondante. Nei sistemi batch, bilancia il carico tra processi CPU-bound e I/O-bound.
- **Scheduler a breve termine (Short-Term Scheduler o CPU Scheduler):** È il componente fondamentale, invocato molto frequentemente (ogni decina di millisecondi). Seleziona, tra i processi nella ready queue (coda dei pronti), quello a cui assegnare la CPU. Deve essere estremamente veloce per non sprecare tempo di CPU.
- **Scheduler a medio termine (Medium-Term Scheduler):** Introduce il concetto di swap (p. 9-10). Quando il grado di multiprogrammazione diventa troppo elevato e le risorse scarseggiano, questo scheduler sospende alcuni processi, spostandoli parzialmente o totalmente dalla memoria principale al disco (area di swap). Questi processi non competono per la CPU e non sono in attesa di I/O; vengono reinseriti in memoria quando le risorse si liberano, per evitare il collasso del sistema.

10.3 2.3 Scheduling con e senza Prelazione (Preemptive vs Non-preemptive) (p. 10)

Le decisioni di scheduling avvengono in quattro circostanze: 1) da esecuzione ad attesa, 2) da esecuzione a pronto, 3) da attesa a pronto, 4) terminazione. Lo scheduling è:

- **Senza prelazione (Non-preemptive o Cooperative):** La CPU viene assegnata a un processo e questi la mantiene fino al suo rilascio volontario (terminazione o attesa per I/O). Semplice da implementare, ma non adatto al time-sharing.
- **Con prelazione (Preemptive):** Un processo può essere forzato a cedere la CPU dallo scheduler (es. per scadenza del quanto di tempo o arrivo di un processo con priorità maggiore). Tutti i sistemi moderni (Windows, Linux) utilizzano la prelazione.

Nota sui kernel (p. 11): Un kernel preemptive richiede meccanismi di sincronizzazione (come i mutex) per proteggere le strutture dati condivise durante l'esecuzione di una chiamata di sistema. Un kernel non-preemptive attende che la chiamata di sistema termini prima di effettuare un cambio di contesto, semplificando la progettazione ma rendendolo inadatto ai sistemi real-time.

10.4 2.4 Il Dispatcher e la Latenza di Dispatch (p. 12)

Il dispatcher è il modulo che si occupa del cambio di contesto effettivo: salva lo stato del processo precedente, carica lo stato del nuovo processo, passa alla modalità utente e salta all'indirizzo del programma per riavviarne l'esecuzione. Il tempo richiesto per queste operazioni è chiamato latenza di dispatch. Per ridurla al minimo, il dispatcher deve essere altamente ottimizzato. È possibile monitorare il numero di cambi di contesto su Linux tramite il comando `vmstat` (colonna `cs`) o visualizzando i dati specifici di un processo (PID) tramite `cat /proc/PID/status`, che distingue tra cambi di contesto volontari (il processo cede la CPU per attendere una risorsa) e involontari (prelazione subita per scadenza del quanto o priorità più alta).

10.5 2.5 Criteri di Scheduling (p. 13-14)

Per confrontare gli algoritmi di scheduling si utilizzano metriche quantitative:

- **Utilizzo della CPU:** Percentuale di tempo in cui la CPU è attiva. Deve essere massimizzata (40% in sistemi leggeri, 90% in carichi pesanti). Visibile con il comando `top`.
- **Produttività (Throughput):** Numero di processi completati per unità di tempo.
- **Tempo di completamento (Turnaround Time):** Intervallo totale dalla sottomissione del processo alla sua terminazione (somma di attese in coda, esecuzione CPU e I/O).
- **Tempo di attesa (Waiting Time):** Somma dei periodi in cui il processo è rimasto nella ready queue in attesa della CPU.
- **Tempo di risposta (Response Time):** Tempo che intercorre dalla sottomissione di una richiesta (es. click del mouse) fino alla prima risposta prodotta (non fino alla fine dell'output). Cruciale per sistemi interattivi.

11 Macro-Capitolo 3 (Thread): Analisi Approfondita degli Algoritmi di Scheduling

11.1 3.1 First-Come, First-Served (FCFS) (p. 15-16)

L'algoritmo più semplice, non preemptive. La CPU viene assegnata al processo che ha richiesto il servizio per primo, gestito tramite una coda FIFO.

- **Vantaggi:** Semplice da implementare.
- **Svantaggi:** Soffre dell'effetto convoglio (convoy effect). Se un processo CPU-bound lungo è in testa alla coda, tutti i processi I/O-bound brevi lo attendono, causando una sotto-utilizzazione della CPU e dei dispositivi di I/O. Il tempo medio di attesa può essere molto elevato. Esempio con P1(24ms), P2(3ms), P3(3ms) in ordine di arrivo: tempo medio di attesa = $(0+24+27)/3 = 17\text{ms}$, che si riduce a 3ms se P2 e P3 fossero eseguiti prima.

11.2 3.2 Shortest-Job-First (SJF) (p. 16-18)

Algoritmo associato a ogni processo la durata del suo prossimo CPU burst; la CPU viene assegnata al processo con il burst più breve.

Ottimalità: Matematicamente, SJF (nella sua versione non preemptive) fornisce il tempo medio di attesa minimo per un dato insieme di processi. Si dimostra che spostando un processo breve davanti a uno lungo, il risparmio del breve supera il danno inflitto al lungo.

Problema pratico: Non è possibile conoscere con certezza la durata del prossimo burst. Si utilizza la media esponenziale (p. 17) per la stima:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

Dove t_n è l'ultimo burst osservato, τ_n è la stima precedente, e α ($0 \leq \alpha \leq 1$) controlla il peso della storia recente (comunemente $\alpha = 0.5$).

Versione con prelazione (Shortest-Remaining-Time-First - SRTF) (p. 18): Se arriva un nuovo processo con un burst stimato più breve del tempo rimanente del processo in esecuzione, il processo corrente viene sottoposto a prelazione. Esempio pratico con processi P1(8ms arrivo 0), P2(4ms arrivo 1), P3(9ms arrivo 2), P4(5ms arrivo 3): il calcolo del tempo medio di attesa (6.5ms) è inferiore alla versione non preemptive (7.75ms).

11.3 3.3 Round Robin (RR) (p. 18-20)

Progettato per sistemi time-sharing. È preemptive e si basa su un quanto di tempo (time slice), tipicamente dai 10 ai 100ms.

Funzionamento: La ready queue è circolare. La CPU viene assegnata al primo processo per un intervallo massimo pari al quanto. Se il processo termina prima, la CPU passa al successivo; altrimenti, scatta un'interruzione, il processo viene spostato in coda alla ready queue e la CPU passa al successivo.

Prestazioni: Se il quanto è troppo grande, RR degenera in FCFS. Se è troppo piccolo, si verifica un numero eccessivo di cambi di contesto, riducendo la produttività. Il turnaround time medio dipende criticamente dalla dimensione del quanto; deve essere almeno un ordine di grandezza maggiore del tempo di cambio di contesto. Esempio con P1(24ms), P2(3ms), P3(3ms) e quanto 4ms mostra un'interleaving che penalizza il tempo di completamento medio rispetto a SJF, ma migliora il tempo di risposta.

11.4 3.4 Scheduling con Priorità (p. 20-22)

Ogni processo ha una priorità (interna, basata su risorse usate, o esterna, basata su importanza). La CPU viene assegnata al processo con priorità più alta; a parità di priorità si usa FCFS.

Problema: Starvation (attesa indefinita). Processi a bassa priorità potrebbero non essere mai eseguiti.

Soluzione: Aging (invecchiamento). Si incrementa gradualmente la priorità dei processi in attesa nel tempo, fino a quando non raggiungono una priorità sufficiente per essere eseguiti.

11.5 3.5 Highest Response Ratio Next (HRRN) (p. 22-23)

Algoritmo non preemptive che mitiga lo starvation di SJF introducendo l'invecchiamento nella formula della priorità.

$$R = 1 + \frac{t_{attesa}}{t_{burst}}$$

Il processo con il più alto response ratio R viene eseguito per primo. Se un processo attende a lungo, il numeratore cresce, aumentandone la priorità, bilanciando la tendenza di SJF a favorire i processi brevi. La priorità viene ricalcolata dinamicamente al termine di ogni burst.

11.6 3.6 Scheduling a Code Multilivello (Multilevel Queue) (p. 23-24)

Le ready queue vengono suddivise in più code separate in base al tipo di processo (es. real-time, interattivi, batch). Ogni coda ha il suo algoritmo di scheduling (es. RR per interattivi, FCFS per batch). Lo scheduler tra le code può essere:

- **A priorità fissa:** La coda con priorità più alta viene servita per prima. Se non è vuota, le code inferiori non vengono toccate.
- **A divisione temporale:** A ogni coda viene assegnata una porzione di tempo CPU (es. 80% alla coda foreground, 20% alla background).

11.7 3.7 Code Multilivello con Retroazione (Multilevel Feedback Queue) (p. 25-26)

È l'algoritmo più flessibile e utilizzato nei sistemi moderni, poiché permette ai processi di migrare tra le code. L'idea è separare i processi in base al loro comportamento (burst lungo o corto).

Esempio funzionale: Tre code Q0 (RR con quanto 8ms), Q1 (RR con quanto 16ms), Q2 (FCFS).

- Un nuovo processo entra sempre in Q0.
- Se in Q0 non termina entro 8ms, viene declassato in Q1.
- Se in Q1 non termina entro 16ms, viene declassato in Q2.
- Q0 ha priorità massima su Q1, che ha priorità su Q2.

Questo meccanismo favorisce i processi I/O-bound (burst brevi, che terminano subito in Q0) e penalizza i CPU-bound (che scendono nelle code inferiori), ottimizzando il tempo di risposta senza richiedere la conoscenza a priori della durata dei burst. Per prevenire la starvation nelle code basse, si utilizza l'aging, spostando periodicamente i processi verso code superiori.

11.8 3.8 Fair Share Scheduling (p. 26-27)

Si discosta dagli algoritmi precedenti che schedulano singoli processi. In contesti in cui un'applicazione è composta da più processi cooperanti, il Fair Share aggrega i processi in gruppi (utenti o applicazioni). La CPU viene suddivisa proporzionalmente ai gruppi (es. 25%, 35%, 40%), e all'interno di ogni gruppo viene applicato un algoritmo di scheduling specifico. Questo garantisce che un utente con molti processi non monopolizzi le risorse a scapito di un utente con pochi processi.

12 Macro-Capitolo 4 (Thread): Metodi di Valutazione e Caso Pratico di Scheduling

12.1 4.1 Valutazione degli Algoritmi (p. 27-30)

Esistono tre metodi principali per valutare le prestazioni di uno scheduler:

- **Modello Deterministico (Deterministic Modeling):** Si prende un carico di lavoro pre-definito e si calcolano le prestazioni di ciascun algoritmo. È semplice e preciso per quel set di dati, ma non generalizzabile. Esempio con 5 processi ($P1=10$, $P2=29$, $P3=3$, $P4=7$, $P5=12$) che mostra come SJF batta FCFS (13ms medi vs 28ms) e RR con quanto 10ms (23ms medi).
- **Modello a Reti di Code (Queueing-Network Analysis):** Si modella il sistema come una rete di server (CPU, I/O) con code di attesa. Conoscendo le distribuzioni statistiche degli arrivi (λ) e dei servizi, si applica la Legge di Little (p. 29) per calcolare il tempo medio di attesa W in relazione alla lunghezza media della coda n e al tasso di arrivo λ : $n = \lambda \times W$. Sebbene potente, è matematicamente complesso e spesso si basa su distribuzioni non realistiche.
- **Simulazione:** Si programma un modello del sistema. Per maggiore accuratezza, si utilizza il Trace Tape: si registra la sequenza degli eventi reali di un sistema in esecuzione e la si usa come input per la simulazione, permettendo un confronto deterministico su un set di dati realistici.

12.2 4.2 Esercizio Pratico Integrato (p. 30-38)

Vengono analizzati cinque processi (A, B, C, D, E) con i seguenti parametri (tempo di arrivo, CPU burst): A(0,3), B(2,6), C(4,4), D(6,5), E(8,2). Di seguito il calcolo di Risposta, Attesa e Turnaround (completamento - arrivo) per ciascun algoritmo.

12.2.1 4.2.1 FCFS (non-preemptive) (p. 31-32)

Ordine: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$. Tutti i calcoli portano a tempi medi di Risposta = 4.6, Attesa = 4.6, Turnaround = 8.6.

12.2.2 4.2.2 SJF non-preemptive (p. 32-33)

Ordine: A (fino a 3) $\rightarrow$ B (fino a 9) $\rightarrow$ poi tra C(4), D(5), E(2) arriva E (burst minore, da 9 a 11), poi C (da 11 a 15), poi D (da 15 a 20). Medie: Risposta = 3.6, Attesa = 3.6, Turnaround = 7.6. Migliore di FCFS.

12.2.3 4.2.3 SJF preemptive (SRTF) (p. 33-34)

Si basa sul tempo rimanente. La sequenza logica produce: A(0-3), B(3-4 e poi 10-15), C(4-8), E(8-10), D(15-20). Medie: Risposta = 2.0, Attesa = 3.2, Turnaround = 7.2.

12.2.4 4.2.4 Round Robin con $q=4$ (p. 35)

Sequenza: A(0-3), B(3-7), C(7-11), D(11-15), B(15-17), E(17-19), D(19-20). Medie: Risposta = 3.6, Attesa = 6.0, Turnaround = 10.0.

12.2.5 4.2.5 Round Robin con $q=1$ (p. 36-37)

Il quanto minimo comporta un'altissima alternanza. La simulazione fornisce medie: Risposta = 0.8 (ottima reattività), Attesa = 6.8, Turnaround = 10.8.

12.2.6 4.2.6 Highest Response Ratio Next (HRRN) (p. 37-38)

L'esercizio mostra due interpretazioni:

- **Opzione 1:** Ricalcolo solo al momento della scelta (senza ricalcolo tra C e D). Ordine A, B, C, D, E. Medie: $R=4.6$, $A=4.6$, $T=8.6$.
- **Opzione 2:** Ricalcolo dinamico rigoroso dopo ogni burst. Al termine di C (tempo 13), si ricalcola per D ($R = 1 + 7/5 = 2.4$) e E ($R = 1 + 5/2 = 3.5$), invertendo l'ordine (E prima di D). Medie: Risposta = 4.0, Attesa = 4.0, Turnaround = 8.0. HRRN si dimostra quindi superiore a FCFS in questo scenario.

13 Macro-Capitolo 5 (Thread): Sincronizzazione tra Processi e la Sezione Critica

13.1 5.1 Race Condition e il Problema del Buffer Limitato (p. 39-40)

Un processo cooperante è un processo che può influenzare o essere influenzato da altri processi in esecuzione. Essi condividono dati tramite spazio di indirizzi comune o file. L'esecuzione concorrente su tali dati genera il problema delle race condition: il risultato finale dell'elaborazione dipende dall'ordine casuale di accesso ai dati condivisi.

L'esempio classico è il produttore-consumatore con un buffer condiviso e una variabile counter. L'istruzione `counter++` in linguaggio ad alto livello si traduce in più istruzioni macchina (load, add, store). Se produttore e consumatore eseguono queste istruzioni in modo interlacciato, il valore finale di counter può essere errato (ad esempio 4 invece di 5). Questa condizione di inconsistenza richiede una sincronizzazione obbligatoria.

13.2 5.2 Il Problema della Sezione Critica (p. 40-42)

In un sistema con n processi, la sezione critica è un segmento di codice in cui il processo accede a risorse condivise (modifica di tabelle, scrittura di file). Il protocollo per gestire l'accesso alla sezione critica deve soddisfare tre requisiti fondamentali:

- **Mutua Esclusione (Mutual Exclusion):** Se un processo P_i è in esecuzione nella sua sezione critica, nessun altro processo può essere in esecuzione nella propria sezione critica.
- **Progresso (Progress):** Se nessun processo è nella sezione critica e alcuni processi desiderano entrarvi, la decisione su chi entrerà per primo deve essere presa solo tra i processi che stanno cercando di entrare (non può essere rimandata all'infinito).
- **Attesa Limitata (Bounded Waiting):** Deve esistere un limite superiore al numero di volte che altri processi possono entrare nella sezione critica dopo che un processo ha richiesto di entrarvi, per prevenire la starvation.

13.3 5.3 Soluzione di Peterson (p. 43-44)

È una soluzione software classica per due processi (P_i e P_j) che condividono due strutture dati: un vettore booleano `flag[2]` (che indica l'intenzione di un processo di entrare nella sezione critica) e una variabile intera `turn` (che indica di chi è il turno).

Protocollo di ingresso: P_i imposta `flag[i] = true` e `turn = j`; poi rimane in attesa (`while(flag[j] && turn == j)`) finché l'altro processo non termina o cede il turno.

Dimostrazione di correttezza: La mutua esclusione è garantita perché se entrambi i processi volessero entrare, solo uno dei due può avere `turn` uguale al proprio indice. Il progresso è assicurato perché se P_j esce, imposta `flag[j]=false`, permettendo a P_i di entrare. L'attesa limitata è rispettata perché P_j può entrare al massimo una volta prima di passare il turno a P_i .

Limite: Non funziona sulle architetture moderne a causa del riordino delle istruzioni (out-of-order execution) da parte del compilatore o della CPU, che potrebbe eseguire le scritture in memoria in un ordine diverso da quello previsto, invalidando la logica.

13.4 5.4 Supporto Hardware per la Sincronizzazione (p. 44-48)

Per ovviare ai limiti delle soluzioni software, le architetture forniscono istruzioni speciali.

Barriere di Memoria (Memory Barriers) (p. 44-45): Istruzioni che forzano il completamento e la visibilità di tutte le operazioni di load e store precedenti, prima di eseguire quelle successive. Utili per garantire che un flag venga letto solo dopo che i dati sono stati scritti, prevenendo race condition su modelli di memoria debolmente ordinati.

Istruzione Test-And-Set (p. 46): Istruzione atomica che legge il valore di una parola di memoria e la imposta a true, restituendo il valore originale. Implementa uno spinlock: `while(test_and_set(&lock))` ;. Il primo processo che chiama la funzione trova `lock=false` e la imposta a true (entra in sezione critica). Gli altri troveranno true e cicleranno (busy waiting).

Istruzione Compare-And-Swap (CAS) (p. 46-48): Istruzione atomica che confronta il valore di una variabile con un valore atteso; se sono uguali, la sostituisce con un nuovo valore; restituisce sempre il valore originale. Può realizzare la mutua esclusione, ma come `test_and_set`, da sola non garantisce l'attesa limitata. Il testo fornisce una versione avanzata di CAS con un vettore `waiting` che soddisfa tutti e tre i requisiti della sezione critica.

Variabili Atomiche (p. 48): Forniscono operazioni atomiche su tipi di dati base (interi, booleani). Utilizzate per contatori condivisi, ma non risolvono race condition su strutture dati più complesse (es. il problema del buffer con due consumatori che leggono lo stesso counter).

14 Macro-Capitolo 6 (Thread): Strumenti di Sincronizzazione Avanzati e Problemi Classici

14.1 6.1 Lock Mutex e Spinlock (p. 49-50)

Il lock mutex è lo strumento software più semplice per implementare la mutua esclusione. Utilizza una variabile booleana `available`.

- **Acquire:** attende in un ciclo `while(!available)` finché il lock non è libero, poi lo imposta a false.

- **Release:** imposta `available = true`.

Busy Waiting (Attesa attiva): Il processo che attende il lock consuma cicli di CPU in un loop. Questo è uno spinlock. È inefficiente in sistemi monoproprocessore (spreca CPU), ma è estremamente utile in sistemi multiprocessore dove il processo può “girare” su un core mentre un altro core esegue la sezione critica, evitando il costoso cambio di contesto. È ideale per sezioni critiche molto brevi.

14.2 6.2 Semafori (p. 50-53)

Un semaforo `S` è una variabile intera a cui si accede solo tramite due operazioni atomiche: `wait()` (o `P`) e `signal()` (o `V`).

- **Semafori Contatori:** Valore intero che rappresenta il numero di risorse disponibili. `wait` decrementa, `signal` incrementa.
- **Semafori Binari:** Valore ristretto a 0 e 1, simile a un mutex ma con una semantica di sveglia più robusta.

Implementazione senza Busy Waiting (p. 52): Si associa a ogni semaforo una coda di processi in attesa. `wait(S)` decrementa il valore; se $S < 0$, il processo si blocca (chiamata `sleep()`) e viene aggiunto alla coda. `signal(S)` incrementa il valore; se $S \leq 0$, viene rimosso un processo dalla coda e riattivato (`wakeup(P)`). In questa implementazione, il valore negativo del semaforo indica il numero di processi in attesa.

Atomicità: Le operazioni su semaforo devono essere atomiche. Su un sistema single-core è sufficiente disabilitare le interruzioni; su sistemi multi-core si richiedono meccanismi hardware come i CAS.

14.3 6.3 Il Problema dei Filosofi a Tavola (p. 53-55)

Problema classico di sincronizzazione: N filosofi alternano pensare e mangiare. Per mangiare, un filosofo ha bisogno di due bacchette (i -esima e $(i+1)$ -esima). Con N bacchette totali, l’approccio naïve di prendere prima la sinistra e poi la destra porta a deadlock se tutti i filosofi prendono contemporaneamente la bacchetta sinistra.

Soluzione presentata (p. 54-55): Si utilizza un vettore di stato `stato[i]` con valori THINKING, HUNGRY, EATING e un semaforo `mutex` per proteggere l’aggiornamento dello stato. Inoltre, c’è un vettore di semafori `f[i]` inizializzati a 0, usati per bloccare singolarmente i filosofi.

- **take_forks(i):** Acquisisce mutex, imposta `stato[i]=HUNGRY` e chiama `test(i)`. Se i vicini non stanno mangiando, `test(i)` imposta `stato[i]=EATING` ed esegue `V(f[i])` (sblocca il filosofo). Altrimenti, il filosofo esegue `P(f[i])` e si blocca.
- **test(i):** Verifica se `stato[i]==HUNGRY` e i vicini `stato[(i+1)%N] != EATING` e `stato[(i-1)%N] != EATING`. Se vero, il filosofo mangia.
- **put_forks(i):** Rilascia il mutex, imposta `stato[i]=THINKING` e chiama `test()` sui due vicini per vedere se possono finalmente mangiare.

Questa soluzione evita il deadlock perché un filosofo mangia solo se i suoi vicini non stanno mangiando, evitando la competizione circolare per tutte le bacchette.

15 LABORATORIO DI SISTEMI OPERATIVI

15.1 Capitolo 6: Concetti Fondamentali (pp. 95 - 104)

Questo capitolo getta le basi teoriche e pratiche per comprendere come un programma viene eseguito, come interagisce con il kernel e come gestisce gli errori.

15.1.1 6.1 Processi e Programmi (p. 95)

- **Definizione di Programma:** È un file binario (eseguibile) che contiene le istruzioni in linguaggio macchina e i dati statici necessari per l'esecuzione.
- **Definizione di Processo:** È l'istanza di un programma in esecuzione. Non è solo il codice, ma include anche lo stato di esecuzione, i dati in memoria e le risorse allocate dal kernel.

Dal punto di vista del kernel, un processo è composto da:

1. **Memoria User Space:** Contiene il codice del programma e le variabili.
2. **Strutture Dati del Kernel:** Mantengono informazioni sullo stato del processo (tabella delle pagine, tabella dei file aperti, segnali in sospeso, limiti delle risorse, ecc.).

6.1.1 Schema della Memoria di un Processo (su Linux/x86-32) (p. 95) Il documento illustra la classica suddivisione della memoria RAM per un processo:

- **Segmento di solo lettura (Program Code / Text):** Contiene le istruzioni in linguaggio macchina. È protetto in scrittura.
- **Segmento Dati Inizializzati (Initialized Data):** Contiene variabili globali e statiche che hanno un valore iniziale esplicito.
- **Segmento Dati Non Inizializzati (Uninitialized Data / BSS):** Contiene variabili globali e statiche che non sono inizializzate (o inizializzate a 0). Il kernel le azzerava prima dell'esecuzione.
- **Heap:** Area per l'allocazione dinamica della memoria (es. con `malloc`). Cresce dal basso verso l'alto (dagli indirizzi più bassi a quelli più alti).
- **Stack:** Area per le variabili locali, gli argomenti delle funzioni e i dati di controllo delle chiamate a funzione. Cresce dall'alto verso il basso (dagli indirizzi più alti a quelli più bassi).

Esempio di codice che copre tutte le sezioni di memoria (p. 96):

```
1 #include <stdlib.h>
2
3 // Dichiarazione di variabili globali
4 char buffer[10]; // <- (bss) Non inizializzata, finisce nel BSS
5 int primes [] = {2, 3, 5, 7}; // <- (data) Inizializzata, finisce nei dati inizializzati
6
7 // Implementazione della funzione
8 void method(int *a) { // <- (stack) L'argomento 'a' e' sullo stack
9     int i; // <- (stack) Variabile locale finisce nello stack
10    for (i = 0 ; i < 10 ; i++) {
11        a[i] = i;
12    }
```

```

13 }
14
15 // Punto di ingresso del programma
16 int main (int argc, char *argv[]) { // <- (stack) argc e argv sono sullo stack
17     static int key = 123; // <- (data) Variabile statica inizializzata finisce nei dati
18     int *p; // <- (stack) Puntatore locale finisce sullo stack
19     p = malloc(10 * sizeof(int)); // <- (heap) Allocazione dinamica, finisce nell'heap
20     method(p);
21     free(p);
22     return 0;
23 }

```

Comando size (p. 96): Per interrogare la dimensione dei segmenti di un eseguibile.

```

programmazionetime@pop-os:~/Scrivania$ size main
text      data      bss      dec      hex      filename
1794      628       24      2446     98e      main

```

- **text:** 1794 byte (codice macchina).
- **data:** 628 byte (dati inizializzati).
- **bss:** 24 byte (dati non inizializzati).
- **dec:** Dimensione totale in decimale (text + data + bss).
- **hex:** Dimensione totale in esadecimale.

Attenzione alla mutabilità delle stringhe (p. 97): Il documento mostra un errore classico.

Codice 1 (ERRORE - Segmentation Fault):

```

1 int main (int argc, char *argv[]) {
2     char *string = "ciao"; // 'string' punta al segmento di solo codice (read-only)
3     string[0] = 'C';        // Tentativo di scrivere in area read-only -> SEGFAULT
4     printf("%s\n", string);
5     return 0;
6 }

```

Motivo: `char *string = "ciao";` crea un puntatore che punta a una stringa letterale, che risiede nel segmento di **codice (read-only)**. Modificarla è illegale.

Codice 2 (CORRETTO):

```

1 int main (int argc, char *argv[]) {
2     char string[] = "ciao"; // Inizializza un array di caratteri sullo STACK
3     string[0] = 'C';        // Modifica valida (lo stack e' scrivibile)
4     printf("%s\n", string);
5     return 0;
6 }

```

Motivo: `char string[] = "ciao";` alloca un array modificabile sullo stack e vi copia la stringa.

6.1.2 Panoramica della Tabella dei Descrittori di File (p. 97) Per ogni processo, il kernel mantiene una **File Descriptor Table**.

- È una tabella che mappa gli interi (file descriptor) alle strutture dei file aperti nel kernel.
- Per convenzione, ogni nuovo processo ha sempre aperti i primi tre descrittori:
 - **0**: Standard Input (STDIN_FILENO)
 - **1**: Standard Output (STDOUT_FILENO)
 - **2**: Standard Error (STDERR_FILENO)
- Quando un processo apre un nuovo file, il kernel assegna il **numero intero più basso disponibile** (di solito il 3, se i primi tre sono già occupati).

15.1.2 6.2 System Calls (Chiamate di Sistema) (p. 98)

Le system call sono funzioni fornite dal kernel che un programma può invocare. Quando un programma chiama una system call, viene generato un **interrupt software** che trasferisce il controllo al kernel. Il kernel esegue l'operazione richiesta in **Kernel Mode** e restituisce il risultato al chiamante.

- Le system call sono documentate nella **sezione 2** del manuale Unix (`man 2 <nome>`).
- Esiste una funzione “wrapper” con lo stesso nome nella libreria standard C (glibc) che prepara i registri e invoca l'interrupt.

6.2.1 Esecuzione delle Chiamate di Sistema (pp. 100-101) Il documento descrive il flusso dettagliato di una system call (es. `execve`):

1. **Applicazione** chiama la funzione wrapper nella libreria C.
2. **Wrapper**:
 - Copia gli argomenti dalla pila (stack) ai registri della CPU.
 - Copia il **numero della system call** (es. `_NR_execve = 11`) nel registro `%eax`.
 - Esegue un'istruzione di **interrupt** (passaggio da User Mode a Kernel Mode).
3. **Kernel**:
 - Riceve l'interrupt.
 - Salva i registri della CPU nello stack del kernel.
 - Usa il numero in `%eax` come indice in una tabella di puntatori a funzioni (`sys_call_table`) per trovare la **System Call Service Routine**.
4. **Routine di Servizio**: Esegue il compito richiesto (es. leggere un file). Restituisce un risultato.
5. **Kernel**:
 - Ripristina i registri della CPU dallo stack del kernel.
 - Salva il risultato del servizio in un registro e commuta la CPU da Kernel Mode a User Mode, tornando al wrapper.
6. **Wrapper**:
 - Se il valore di ritorno del kernel indica un errore, imposta la variabile globale `errno`.

- Ritorna al programma chiamante un valore intero (0 o -1 per indicare successo o fallimento).

Convenzione: In caso di errore, la system call restituisce **-1** (o NULL per i puntatori) e imposta **errno**.

6.2.2 Gestione degli Errori delle Chiamate di Sistema (pp. 101-102) La variabile globale **errno** (dichiarata in `<errno.h>`) viene impostata con un valore specifico che identifica l'errore. Le costanti simboliche per gli errori (es. **EACCES**, **ENOENT**) sono definite in `<errno.h>`.

Esempio di base con errno (p. 101):

```
1 #include <errno.h>
2
3 // Apertura di un file
4 fd = open(pathname, flags, mode);
5
6 // INIZIO della gestione degli errori
7 if (fd == -1) {
8     if (errno == EACCES) {
9         // La gestione non ha consentito l'accesso al file
10    } else {
11        // Altri errori
12    }
13 }
14 // FINE della gestione degli errori
```

Caso particolare: Per system call che restituiscono -1 anche in caso di successo (es. **getpriority**), si deve impostare **errno = 0** prima della chiamata e controllare dopo.

```
1 #include <sys/resource.h>
2
3 // Imposta il valore di errno a 0
4 errno = 0;
5
6 // La chiamata getpriority prende il valore nice di un processo
7 nice = getpriority(which, who);
8 if ((nice == -1) && (errno != 0)) {
9     // Gestione dell'errore
10 }
```

Funzione perror (p. 101): Stampa su **stderr** la stringa passata, seguita da “:” e dal messaggio di errore corrispondente a **errno**.

```
1 #include <stdio.h>
2 void perror(const char *msg);
```

Esempio con perror (p. 101):

```
1 #include <stdio.h>
2
3 // Apertura di un file
```

```

4 fd = open(pathname, flags, mode);
5 if (fd == -1) {
6     perror("<Open>");
7     // Uccide il processo corrente
8     exit(EXIT_FAILURE);
9 }

```

Output: <Open>: No such file or directory

Funzione `strerror` (p. 101): Prende un numero di errore (`errno`) e restituisce una stringa descrittiva. È definita in `<string.h>`.

```

1 #include <string.h>
2 char *strerror(int errnum);

```

Esempio con `strerror` (p. 101):

```

1 #include <stdio.h>
2
3 // Apertura di un file
4 fd = open(path, flags, mode);
5 if (fd == -1) {
6     printf("Error opening (%s):\n\t%s\n", path, strerror(errno));
7     // Uccide il processo corrente
8     exit(EXIT_FAILURE);
9 }

```

Output: Error opening (myFile.txt): No such file or directory

Funzione helper `errExit` (p. 102): Non è una funzione standard C, ma è comunemente definita come wrapper.

```

1 void errExit(const char *msg) {
2     perror(msg);
3     exit(EXIT_FAILURE);
4 }

```

6.2.3 Comando `strace` (p. 102) Il comando `strace` viene utilizzato per tracciare le system call effettuate da un processo. È estremamente utile per il debugging e per capire cosa fa un programma a basso livello.

`strace <comando> <argomenti>`

(Esempio visivo a p. 102 per `strace ls /tmp`)

15.1.3 6.3 Tipi di Dato del Kernel (p. 102)

Il kernel utilizza tipi di dato specifici per garantire portabilità su diverse architetture. Sono definiti con `typedef` in `<sys/types.h>` e spesso terminano con `_t`.

(Tabella a p. 103)

Tipo	Contenuto
------	-----------

<code>caddr_t</code>	Core address
<code>clock_t</code>	Contatore del process time
<code>dev_t</code>	Numero di dispositivo
<code>gid_t</code>	Identificatore di un gruppo
<code>ino_t</code>	Numero di inode
<code>key_t</code>	Chiave per il System V IPC
<code>off_t</code>	Posizione corrente in un file
<code>mode_t</code>	Attributi di un file
<code>nlink_t</code>	Contatore dei collegamenti su un file
<code>pid_t</code>	Identificatore di un processo
<code>rlim_t</code>	Limite sulle risorse
<code>sigset_t</code>	Insieme di segnali
<code>size_t</code>	Dimensione di un oggetto (senza segno)
<code>ssize_t</code>	Dimensione in byte (con segno, per funzioni)
<code>ptrdiff_t</code>	Differenza tra due puntatori
<code>time_t</code>	Numero di secondi
<code>uid_t</code>	Identificatore di un utente
<code>msgqnum_t</code>	Numero di messaggi nella coda
<code>msglen_t</code>	Numero di byte concessi per un messaggio
<code>shmatt_t</code>	Conteggio allegati per memoria condivisa

15.1.4 6.4 Pagine di Manuale (p. 103)

Il comando `man` fornisce la documentazione.

- Sezione 1: Comandi utente.
- Sezione 2: System calls.
- Sezione 3: Librerie di funzione (C standard).
- Sezione 4: Dispositivi.
- Sezione 5: Formati file.
- `man -k <stringa>`: Cerca nel database delle pagine di manuale.

15.2 Capitolo 7: Gestore dei File e Directory (pp. 104 - 127)

Questo capitolo spiega la filosofia di Linux (“everything is a file”) e fornisce la guida completa alle system call per la gestione dei file.

15.2.1 Introduzione: Virtual File System (VFS) (p. 104)

Linux astrae tutti i dispositivi di I/O tramite file. Il **VFS** è uno strato intermedio nel kernel che fornisce un’interfaccia uniforme alle system call di I/O, indipendentemente dal filesystem sottostante (ext4, NTFS, NFS, ecc.).

- Il processo chiama una system call.
- Il kernel chiama la funzione corrispondente nel **VFS**.

- Il VFS utilizza le funzioni specifiche del **filesystem** montato.
- Il filesystem specifico gestisce le operazioni di I/O sul **dispositivo fisico**.

(Schema VFS a p. 105)

15.2.2 7.1 File

7.1.1 Operazione di Apertura/Creazione: open (p. 106) La system call **open** apre un file esistente o ne crea uno nuovo.

Prototipi:

```
1 #include <sys/type.h>
2 #include <sys/stat.h>
3 #include <fcntl.h>
4
5 int open(const char *pathname, int flags);
6 int open(const char *pathname, int flags, mode_t mode);
```

- **Ritorno:** File descriptor (intero ≥ 3) in caso di successo, **-1** in caso di errore (con **errno** impostato).
- **flags:** Specifica la modalità di accesso (lettura, scrittura, ecc.) e opzioni di comportamento.
- **mode:** Specifica i permessi del file se viene creato (**O_CREAT**). Viene applicato dopo la maschera **umask**.

Tabella dei Flag (p. 107):

Flag	Significato
O_RDONLY	Apre il file in sola lettura.
O_WRONLY	Apre il file in sola scrittura.
O_RDWR	Apre il file sia in lettura sia in scrittura.
O_TRUNC	Tronca il file esistente a lunghezza zero (se esiste e se permesso).
O_APPEND	Le scritte vengono sempre aggiunte alla fine del file.
O_CREAT	Se il file non esiste, viene creato.
O_EXCL	Usato con O_CREAT ; fallisce se il file esiste già (utile per creazione atomica).
O_DIRECTORY	Fallisce se pathname non è una directory.
O_TMPFILE	Crea un file temporaneo anonimo (non visibile nel filesystem).
O_NOFOLLOW	Fallisce se pathname è un collegamento simbolico.
O_PATH	Ottiene un fd limitato per operazioni sul filesystem.
O_NONBLOCK	Apre il file in modalità non bloccante per I/O.
O_NOATIME	Blocca l'aggiornamento del tempo di accesso del file.

Tabella dei Mode (permessi) (p. 108):

Flag	Significato
------	-------------

S_IWXU	L'utente ha permessi di lettura, scrittura ed esecuzione.
S_IRUSR	L'utente ha permesso di lettura.
S_IWUSR	L'utente ha permesso di scrittura.
S_IXUSR	L'utente ha permesso di esecuzione.
S_IWXG	Il gruppo ha permessi di lettura, scrittura ed esecuzione.
S_IRGRP	Il gruppo ha permesso di lettura.
S_IWGRP	Il gruppo ha permesso di scrittura.
S_IXGRP	Il gruppo ha permesso di esecuzione.
S_IWXX	Gli altri hanno permessi di lettura, scrittura ed esecuzione.
S_IROTH	Gli altri hanno permesso di lettura.
S_IWOTH	Gli altri hanno permesso di scrittura.
S_IXOTH	Gli altri hanno permesso di esecuzione.

La umask (p. 108): I permessi finali di un file appena creato sono dati da `mode & ~umask`. Il valore predefinito della `umask` è spesso 022, che rimuove i permessi di scrittura per il gruppo e gli altri.

Esempi di open (pp. 109-110):

```
1 int fd;
2 // Apertura del file esistente in modalita' solo scrittura.
3 fd = open("myfile", O_WRONLY);

4
5 // Apertura di un nuovo file esistente per lettura/scrittura
6 // troncamento a 0 byte; permessi ai file lettura + scrittura
7 // solo per il proprietario.
8 fd = open("myfile1", O_RDWR | O_CREAT | O_TRUNC, S_IRUSR | S_IWUSR);

9
10 // Crea e apre un nuovo file per lettura/scrittura
11 // permessi dei file lettura + scrittura solo per il proprietario.
12 fd = open("myfile2", O_RDWR | O_CREAT | O_EXCL, S_IRUSR | S_IWUSR);
```

7.1.2 Operazione di Lettura: read (p. 110) Legge dati da un file descriptor.

Prototipo:

```
1 #include <unistd.h>
2
3 ssize_t read(int fd, void *buf, size_t count);
```

- **Ritorno:** Numero di byte letti in caso di successo (**0** per EOF), **-1** in caso di errore.
- Tenta di leggere `count` byte da `fd` nella posizione corrente, spostando il cursore in avanti.
- **Nota:** Può restituire meno byte di quelli richiesti (non è un errore). Ciò accade se si raggiunge l'EOF, o se si leggono da pipe/socket dove i dati arrivano a blocchi.

Esempio 1: Leggere da un file (p. 111):

```
1 // Apertura di un file esistente in modalita' sola lettura.
```

```

2 int fd = open("myfile", O_RDONLY);
3 if (fd == -1)
4     errExit("open");
5
6 // Un MAX_READ di byte per il buffer.
7 char buffer[MAX_READ + 1];
8
9 // Si leggono fino a MAX_READ byte dal myfile.
10 ssize_t numRead = read(fd, buffer, MAX_READ);
11 if (numRead == -1)
12     errExit("read");

```

Nota: In un file regolare, `read` restituisce 0 per segnalare la fine del file (EOF).

Esempio 2: Leggere dal terminale (STDIN) (p. 111):

```

1 // Un MAX_READ di byte per il buffer.
2 char buffer[MAX_READ + 1];
3
4 // Si leggono fino a MAX_READ byte da STDIN.
5 ssize_t numRead = read(STDIN_FILENO, buffer, MAX_READ);
6 if (numRead == -1)
7     errExit("read");
8
9 buffer[numRead] = '\0';
10 printf("Input data: %s\n", buffer);

```

Nota: Quando si legge da un terminale, `read` restituisce i caratteri letti fino al carattere di newline (`\n`).

7.1.3 Operazione di Scrittura: write (p. 111) Scrive dati su un file descriptor.

Prototipo:

```

1 #include <unistd.h>
2
3 ssize_t write(int fd, void *buf, size_t count);

```

- **Ritorno:** Numero di byte scritti in caso di successo, **-1** in caso di errore.
- Se il file è aperto con `O_APPEND`, i dati vengono sempre scritti alla fine.

Esempio 1: Scrivere su un file (p. 112):

```

1 // Apertura di un file esistente in modalita' sola scrittura.
2 int fd = open("myfile", O_WRONLY);
3 if (fd == -1)
4     errExit("open");
5
6 // Un buffer memorizza la stringa.
7 char buffer[] = "Ciao, mondo";
8
9 // Si scrive fino a sizeof(buffer) byte all'interno di myfile.

```

```

10 ssize_t numWrite = write(fd, buffer, sizeof(buffer));
11 if (numWrite != sizeof(buffer))
12     errExit("write");

```

Esempio 2: Scrivere su STDOUT (p. 113):

```

1 // Un buffer memorizza una stringa.
2 char buffer[] = "Ciao, mondo";
3
4 // Si scrive fino a sizeof(buffer) byte su STDOUT.
5 ssize_t numWrite = write(STDOUT_FILENO, buffer, sizeof(buffer));
6 if (numWrite != sizeof(buffer))
7     errExit("write");

```

7.1.4 Gestione della Posizione sul File: lseek (p. 113) Modifica la posizione corrente del file descriptor.

Prototipo:

```

1 #include <sys/types.h>
2 #include <unistd.h>
3 off_t lseek(int fd, off_t offset, int whence)

```

- **Ritorno:** Nuova posizione (offset) in caso di successo, **-1** in caso di errore.
- Non causa accesso al file, modifica solo il cursore nel kernel.

Tabella whence (p. 113):

Costante	Significato
SEEK_SET	Offset assoluto dall'inizio del file (offset deve essere ≥ 0).
SEEK_CUR	Offset relativo alla posizione corrente (può essere negativo).
SEEK_END	Offset relativo alla fine del file (può essere negativo).
SEEK_DATA	Posiziona all'inizio del primo blocco di dati dopo un buco.
SEEK_HOLE	Posiziona all'inizio del primo buco nel file.

Esempi di lseek (p. 114):

```

1 // Primo byte del file.
2 off_t current = lseek(fd1, 0, SEEK_SET);
3 // Ultimo byte del file.
5 off_t current = lseek(fd2, -1, SEEK_END);
8 off_t current = lseek(fd3, -10, SEEK_CUR);
9 // 10imo byte dopo la posizione di offset corrente del file.
11 off_t current = lseek(fd4, 10, SEEK_CUR);

```

7.1.5 Chiusura di un File Descriptor: close (p. 114) Rilascia un file descriptor.

Prototipo:

```

1 #include <unistd.h>
2
3 int close(int fd)

```

- **Ritorno:** 0 in caso di successo, **-1** in caso di errore.
- Rilascia eventuali blocchi sul file. Se `fd` è l'ultimo riferimento, tutte le risorse del kernel vengono liberate.
- **Attenzione:** Quando un processo termina, tutti i file descriptor vengono chiusi automaticamente (ma è buona norma chiamare `close` esplicitamente e controllare l'errore, perché una `write` può avere successo anche se i dati non sono ancora stati fisicamente scritti su disco).

7.1.6 Rimozione di un File: `unlink` (p. 116) Elimina il nome di un file dal filesystem. Il file viene cancellato fisicamente solo quando il contatore dei collegamenti (hard link) arriva a zero.

Prototipo:

```
1 #include <unistd.h>
2 int unlink(const char *pathname)
```

- **Ritorno:** 0 in caso di successo, **-1** in caso di errore.

Esempio (p. 116):

```
1 // Creazione di un nuovo file chiamato myFile
2 int fd = open("myFile", O_CREAT | O_WRONLY);
3 // ... scrive solo quando myFile e' aperto in sola scrittura
4 // Chiusura del file descriptor fd
5 close(fd);
6
7 // Scollegamento (remove) myFile
8 unlink("myFile");
```

Nota: Spesso si usa `unlink` subito dopo aver aperto un file temporaneo, per garantire che venga eliminato alla chiusura del processo, senza bisogno di un intervento esplicito.

15.2.3 7.2 Caratteristiche dei File

7.2.1 Lettura delle Caratteristiche: `stat`, `lstat`, `fstat` (p. 117) Ottenere informazioni su un file.

Prototipi:

```
1 #include <sys/type.h>
2 #include <sys/stat.h>
3 #include <unistd.h>
4
5 int stat(const char *pathname, struct stat *statbuf);
6 int lstat(const char *pathname, struct stat *statbuf);
7 int fstat(int fd, struct stat *statbuf);
```

- `stat`: Segue i link simbolici (legge le info del file puntato).
- `lstat`: NON segue i link simbolici (legge le info del link stesso).
- `fstat`: Legge le info di un file già aperto (tramite `fd`).

Struttura `stat` (p. 117):

```

1 struct stat {
2     dev_t      st_dev;      // ID di un dispositivo contenente un file
3     ino_t      st_ino;      // Numero di inode
4     mode_t     st_mode;     // Tipo di file e permessi
5     nlink_t    st_nlink;    // Numero di collegamenti al file
6     uid_t      st_uid;      // User ID del proprietario
7     gid_t      st_gid;      // Group ID del proprietario
8     dev_t      st_rdev;     // ID per file speciali di dispositivi
9     off_t      st_size;     // Dimensione in byte del file
10    blksize_t   st_blksize;  // Dim. in byte ottimale del blocco per I/O
11    blkcnt_t    st_blocks;   // Numero di blocchi allocati da 512B
12    time_t      st_atime;    // Ultimo tempo d'accesso
13    time_t      st_mtime;    // Ultimo tempo di modifica
14    time_t      st_ctime;    // Ultimo tempo di cambio di stato
15 };

```

- **st_nlink**: Hard link.
- **st_blocks**: Spazio effettivo allocato su disco (differisce da **st_size**).

7.2.2 Tipi di File e Permessi (p. 118) Il campo **st_mode** è una maschera binaria. I bit si dividono in:

- Bit per il **tipo di file**.
- Bit per i **permessi** (User, Group, Other).
- Bit speciali: SUID, SGID, Sticky.

Macro per identificare il tipo di file (p. 119):

Macro	Tipo di file
S_ISREG(m)	File normale
S_ISDIR(m)	Directory
S_ISCHR(m)	Dispositivo a caratteri
S_ISBLK(m)	Dispositivo a blocchi
S_ISFIFO(m)	FIFO
S_ISLNK(m)	Collegamento simbolico
S_ISSOCK(m)	Socket

Costanti per la maschera (p. 119):

Costante	Valore	Significato
S_IFMT	0170000	Maschera per i bit del tipo di file
S_IFSOCK	0140000	Socket
S_IFLNK	0120000	Collegamento simbolico
S_IFREG	0100000	File regolare
S_IFDIR	0040000	Directory
S_IFCHR	0020000	Dispositivo a caratteri
S_IFIFO	0010000	FIFO

S_ISUID	0004000	Set user ID bit (SUID)
S_ISGID	0002000	Set group ID bit (SGID)
S_ISVTX	0001000	Sticky bit
S_IRWXU	00700	Maschera permessi proprietario
S_IRUSR	00400	Lettura proprietario
S_IWUSR	00200	Scrittura proprietario
S_IXUSR	00100	Esecuzione proprietario
S_IRWXG	00070	Maschera permessi gruppo
S_IRGRP	00040	Lettura gruppo
S_IWGRP	00020	Scrittura gruppo
S_IXGRP	00010	Esecuzione gruppo
S_IRWXO	00007	Maschera permessi altri
S_IROTH	00004	Lettura altri
S_IWOTH	00002	Scrittura altri
S_IXOTH	00001	Esecuzione altri

Esempio di controllo del tipo di file (p. 120):

```

1 char pathname[] = "/tmp/file.txt";
2 struct stat statbuf;
3 // Si assumono gli attributi di /tmp/file.txt
4 if (stat(pathname, &statbuf) == -1)
5     errExit("stat");
6
7 // Si controlla se /tmp/file.txt e' un file regolare.
8 if ((statbuf.st_mode & S_IFMT) == S_IFREG)
9     printf("regular file!\n");
10
11 // Equivalente, si controlla /tmp/file.txt se e' un
12 // file regolare dalla macro S_ISREG.
13 if (S_ISREG(statbuf.st_mode))
14     printf("regular file!\n");

```

7.2.4 Controllare l'Accessibilità ai File: access (p. 121) Verifica i permessi di accesso utilizzando l'**UID** e **GID** reali del processo (non quelli effettivi). Utile per verificare se un utente reale ha determinati permessi su un file.

Prototipo:

```

1 #include <unistd.h>
2
3 int access(const char *pathname, int mode);

```

- mode: R_OK (lettura), W_OK (scrittura), X_OK (esecuzione), F_OK (esistenza).

7.2.5 Cambiare i Permessi ai File: chmod, fchmod (p. 122) Modifica i permessi di un file.

Prototipi:

```

1 #include <sys/stat.h>

```

```

2
3 int chmod(const char *pathname, mode_t mode);
4 int fchmod(int fd, mode_t mode);

```

Esempio (p. 123):

```

1 char pathname[] = "/tmp/file.txt";
2
3 struct stat sb;
4 if (stat(pathname, &sb) == -1)
5     errExit("stat");
6
7 // Attiva la scrittura del proprietario,
8 // Disattiva la lettura degli altri,
9 // i rimanenti bit sono invariati.
10 mode_t mode = (sb.st_mode | S_IWUSR) & ~S_IROTH;
11
12 if (chmod(pathname, mode) == -1)
13     errExit("chmod");

```

15.2.4 7.3 Directory

7.3.1 Creazione di una Directory: mkdir (p. 123) Prototipo:

```

1 #include <sys/stat.h>
2 #include <sys/types.h>
3
4 int mkdir(const char *dirname, mode_t mode)

```

Crea una directory con i permessi specificati da mode (soggetti alla umask).

7.3.2 Rimozione di una Directory: rmdir (p. 124) Prototipo:

```

1 #include <unistd.h>
2
3 int rmdir(const char *pathname);

```

La directory deve essere vuota (contenere solo "." e "..") e non essere in uso.

Esempio combinato (p. 124):

```

1 // Si crea una nuova directory con il nome myDir
2 int res = mkdir("myDir", S_IRUSR | S_IXUSR);
3 if (res == 0) {
4     printf("The directory myDir was created!\n");
7     res = rmdir("myDir");
8     if (res == 0)
9         printf("The directory myDir was removed!\n");
10 }

```

7.3.3 Apertura e Chiusura di una Directory: opendir, closedir (p. 125) Per leggere il contenuto di una directory, POSIX definisce i **directory stream**.

opendir:

```
1 #include <sys/types.h>
2 #include <dirent.h>
3 DIR *opendir(const char *dirpath);
```

closedir:

```
1 #include <sys/types.h>
2 #include <dirent.h>
3 int closedir(DIR *dir)
```

7.3.4 Lettura di una Directory: readdir (p. 125) Prototipo:

```
1 #include <sys/types.h>
2 #include <dirent.h>
3 struct dirent *readdir(DIR *dirp);
```

Restituisce un puntatore a una struttura `dirent` per la voce corrente, spostandosi alla successiva. Restituisce NULL quando non ci sono più voci (o in caso di errore; per distinguere, si controlla `errno`).

Struttura dirent (p. 126):

```
1 struct dirent {
2     ino_t d_ino;           // Numero di inode
3     off_t d_off;           // Offset alla successiva dirent
4     unsigned short int d_reclen; // Lunghezza di questo record
5     unsigned char d_type;   // Tipo di file
6     char d_name [256];     // Nome del file
7 };
```

Tabella d_type (p. 126):

Valore	Tipo di file
DT_UNKNOWN	Tipo sconosciuto
DT_REG	File normale
DT_DIR	Directory
DT_LNK	Collegamento simbolico
DT_FIFO	FIFO
DT_SOCKET	Socket
DT_CHR	Dispositivo a caratteri
DT_BLK	Dispositivo a blocchi

Esempio: Visualizzare solo i file regolari in una directory (p. 127):

```
1 DIR *dp = opendir("myDir");
2 if (dp == NULL) return -1;
3
4 errno = 0;
5 struct dirent *dentry;
6
7 // Si itera finche' non viene ritornato NULL come risultato.
```

```

8 while ( (dentry = readdir(dp)) != NULL ) {
9     if (dentry->d_type == DT_REG) {
10         printf("Regular file: %s\n", dentry->d_name);
11     }
12     errno = 0;
13 }
14 // Si ritorna NULL su error quando si raggiunge la fine della directory.
15 if (errno != 0) {
16     printf("Error while reading dir.\n");
17 }
18 closedir(dp);

```

15.3 Capitolo 8: Gestione dei Processi (pp. 128 - 156)

Questo capitolo copre come i processi vengono creati, identificati, gestiti e come eseguono nuovi programmi.

15.3.1 8.1 Attributi di un Processo

8.1.1 Identificatori dei Processi: PID e PPID (p. 129)

- **PID (Process ID):** Numero univoco per ogni processo (tipo `pid_t`, intero con segno a 16/32 bit). Il processo `init` ha sempre PID 1. I PID vengono riutilizzati dopo aver raggiunto il massimo (es. 32768).
- **PPID (Parent Process ID):** PID del processo che ha creato il processo corrente.

Funzioni:

```

1 #include <sys/types.h>
2 #include <unistd.h>
3
4 pid_t getpid(void); // Restituisce il PID del processo corrente.
5 pid_t getppid(void); // Restituisce il PID del processo padre.

```

8.1.2 Reale ed Effettivo PID (UID/GID) (p. 130) Per il controllo di accesso, i processi hanno diversi identificatori:

Suffisso	Gruppo	Significato
uid	Real UID	Utente che ha avviato il programma.
gid	Real GID	Gruppo principale dell'utente.
euid	Effective UID	Utente usato per il controllo di accesso ai file.
egid	Effective GID	Gruppo usato per il controllo di accesso ai file.
suid/sgid	Saved	Copia dell'Effective ID iniziale (usato da <code>setuid</code>).
fsuid/fsgid	Filesystem	Usato specificamente per l'accesso al filesystem.

- Normalmente Real ID = Effective ID. Cambiano quando un programma ha il **SUID bit** o **SGID bit** impostato, permettendo a un utente di ottenere i permessi del proprietario del file.

Funzioni per leggere gli identificatori (p. 131):

```
1 #include <unistd.h>
2 #include <sys/types.h>
3
4 uid_t getuid(void);    // Legge l'UID reale.
5 uid_t geteuid(void);   // Legge l'UID effettivo.
6 gid_t getgid(void);    // Legge il GID reale.
7 gid_t getegid(void);   // Legge il GID effettivo.
```

Esempio di stampa (p. 131):

```
1 #include <unistd.h>
2 #include <sys/types.h>
3 int main (int argc, char *argv[]) {
4     printf("PID: %d, user-ID: real %d, effective %d\n", getpid(), getuid(), geteuid());
5     return 0;
6 }
```

8.1.3 Variabili d'Ambiente (p. 131) Ogni processo riceve una lista di variabili d'ambiente (environment list), che è un array di puntatori a stringhe terminato da NULL.

Accesso tramite environ (p. 132):

```
1 #include <stdio.h>
2
3 // Variable globale all'ambiente del processo.
4 extern char **environ;
5
6 int main(int argc, char *argv[]) {
7
8     for (char **it = environ ; (*it) != NULL ; it++) {
9         printf("--> %s\n", *it);
10    }
11
12    return 0;
13 }
```

Output esempio:

```
--> $HOME=/home/Professor
--> $PWD=/tmp
--> $USER=Professor
```

Accesso tramite terzo argomento di main (p. 132):

```
1 #include <stdio.h>
2
3 int main(int argc, char *argv[], char *env[]) {
4
```

```

5     for (char **it = env ; (*it) != NULL ; it++) {
6         printf("--> %s\n", *it);
7     }
8
9     return 0;
10 }

```

Funzioni per gestire l'ambiente (p. 133):

- `getenv`: Cerca il valore di una variabile.

```

1 #include <stdlib.h>
2 char *getenv(const char *name);

```

- `setenv`: Aggiunge o modifica una variabile.

```

1 #include <stdlib.h>
2 int setenv(const char *name, const char *value, int overwrite);

```

- Se `overwrite != 0`, sovrascrive.
- Se `overwrite == 0`, non modifica.

- `unsetenv`: Rimuove una variabile.

```

1 #include <stdlib.h>
2 int unsetenv(const char *name);

```

8.1.4 Directory di Lavoro (p. 134) Ogni processo ha una directory corrente (CWD).

- `getcwd`: Ottiene il pathname assoluto della CWD.

```

1 #include <unistd.h>
2 char *getcwd(char *buffer, size_t size);

```

- `chdir` / `fchdir`: Cambia la CWD.

```

1 #include <unistd.h>
2 int chdir(const char *pathname);
3 int fchdir(int fd);

```

Esempio completo con `chdir` e `fchdir` (p. 135):

```

1 char buf [PATH_MAX];
2
3 // Si apre la directory di lavoro corrente
4 int fd = open("../", O_RDONLY);
5 getcwd(buf, PATH_MAX);
6 printf("1) Current dir:\n\t%s\n", buf);
7
8 // Si muove il processo all'interno di /tmp
9 chdir("/tmp");
10 getcwd(buf, PATH_MAX);
11 printf("2) Current dir:\n\t%s\n", buf);
12

```

```

13 // Si muove il processo all'indietro nella directory iniziale
14 fchdir(fd);
15 getcwd(buf, PATH_MAX);
16 printf("3) Current dir:\n\t%s\n", buf);
17
18 // Si chiude il file descriptor
19 close(fd);

```

Output:

```

1) Current dir: /home/Professor
2) Current dir: /tmp
3) Current dir: /home/Professor

```

8.1.5 Tabella dei Descrittori di File e dup (p. 136) La directory `/proc/<PID>/fd` mostra i link simbolici ai file aperti dal processo.

Reindirizzamento di STDOUT (p. 136):

```

1 // Si chiude STDOUT avente FD 1. I rimanenti file descriptors hanno
2 // indice 0 (stdin) e 2 (stderr).
3 close(STDOUT_FILENO);
4
5 // Si apre un nuovo file, per il quale si assegna FD 1 automaticamente
6 // perche' e' il piu' piccolo indice disponibile nella tabella.
7 int fd = open("myfile", O_TRUNC | O_CREAT | O_WRONLY, S_IRUSR | S_IWUSR);
8 // Printf utilizza FD 1, in tal modo stampera' sul file.
9 printf("ciao\n");

```

Risultato: printf scrive su myfile, non sul terminale.

Duplicazione di file descriptor: dup (p. 136)

Crea una copia di un file descriptor. Entrambi puntano alla stessa voce nella tabella dei file del kernel, condividendo posizione e flag.

```

1 #include <unistd.h>
2 int dup(int oldfd);

```

Esempio di duplicazione per reindirizzare STDOUT e STDERR (p. 137):

```

1 // FDT: [0, 1, 2] -> [0, 2]
2 close(STDOUT_FILENO);
3
4 // FDT: [0, 2] -> [0]
5 close(STDERR_FILENO);
6 // FDT: [0] -> [0, 1]
7 int fd = open("myfile", O_TRUNC | O_CREAT | O_WRONLY, S_IRUSR | S_IWUSR);
8
9 // FDT: [0, 1] -> [0, 1, 2]
10 dup(1);
11
12 // FDT: [0: STDIN, 1: myfile, 2: myfile]

```

```

13 printf("Have a good ");
14
15 fflush(stdout);
16 fprintf(stderr, "day!\n");

```

Output (scritto nel file): Have a good day!

15.3.2 8.3 Operazioni con i Processi

8.3.1 Terminazione di un Programma: `exit` e `_exit` (p. 136)

- `exit(int status)`: Terminazione ordinaria. Esegue le funzioni registrate con `atexit`, chiude gli stream C (flushatura) e chiama `_exit`.
- `_exit(int status)`: Terminazione immediata. Passa il controllo al kernel senza flushatura e senza eseguire `atexit`.

Convenzione: 0 per successo, valori 1-255 per fallimento.

8.3.2 Esecuzione di Funzioni Preliminari all'Uscita: `atexit` (p. 137) Registra una funzione che verrà eseguita automaticamente alla terminazione del processo (tramite `exit` o `return` da `main`).

```

1 #include <stdlib.h>
2 int atexit(void (*function)(void));

```

Esempio (p. 138):

```

1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 void func1(void) {
6     printf("\tAtexit function 1 called\n");
7 }
8
9 void func2(void) {
10    printf("\tAtexit function 2 called\n");
11 }
12
13 int main(int argc, char *argv[]) {
14
15     if (atexit(func1) != 0 || atexit(func2) != 0) {
16         _exit(EXIT_FAILURE);
17     }
18
19     exit(EXIT_SUCCESS);
20 }

```

Output (le funzioni vengono eseguite in ordine LIFO - ultima registrata prima):

```

Atexit function 2 called
Atexit function 1 called

```

8.3.3 Creazione dei Processi: fork (p. 138) La `fork` è l'unico modo per creare un nuovo processo. Crea un processo figlio che è una copia quasi identica del padre.

```
1 #include <unistd.h>
2 pid_t fork(void);
```

- **Ritorno nel padre:** PID del figlio.
- **Ritorno nel figlio:** 0.
- **Errore:** -1.

Caratteristiche:

- Il figlio esegue dall'istruzione successiva alla `fork`.
- Il figlio riceve copie della memoria del padre (stack, heap, dati). Le modifiche alla memoria sono indipendenti.

Esempio 1 (p. 139):

```
1 #include <unistd.h>
2
3 int main(int argc, char *argv[]) {
4     int stack = 111;
5     pid_t pid = fork();
6
7     if (pid == -1) {
8         errExit("fork");
9     }
10
11     // Sia il genitore che figlio partono da qui
12     if (pid == 0) {
13         stack = stack * 4;
14     }
15
16     printf("\t%s stack %d\n", (pid==0) ? "(child )" : "(parent)", stack);
17 }
```

Output:

```
(parent) stack 111
(child ) stack 444
```

(I due processi hanno variabili `stack` indipendenti).

Esempio 2: PID e PPID dopo fork (p. 140):

```
1 #include <unistd.h>
2
3 int main(int argc, char *argv[]) {
4     pid_t pid = fork();
5
6     if (pid == -1) {
7         errExit("fork");
```

```

8     }
9
10    if (pid == 0) {
11        printf("(child ) PID: %d PPID: %d\n", getpid(), getppid());
12    } else {
13        printf("(parent) PID: %d PPID: %d\n", getpid(), getppid());
14    }
15    return 0;
16 }

```

Possibili scenari:

1. Figlio esegue dopo padre (normale): padre PPID=350, PID=402; figlio PID=403, PPID=402.
2. Figlio esegue prima del padre: l'output può mescolarsi, ma i valori sono corretti.
3. Padre termina prima del figlio: il figlio viene **adottato** da init (PID 1), quindi il suo PPID diventa 1.

8.3.4 Monitorare i Figli di un Processo: wait e waitpid (p. 141) Per evitare processi **zombie** (processi terminati che non vengono raccolti dal padre), si usano **wait** e **waitpid**.

wait (p. 141): Sospende il padre fino a quando un qualsiasi figlio termina.

```

1 #include <sys/types.h>
2 #include <sys/wait.h>
3 pid_t wait(int *status);

```

Ritorna il PID del figlio terminato, o -1 in caso di errore (es. ECHILD).

waitpid (p. 143): Più flessibile, aspetta un figlio specifico o con opzioni.

```

1 #include <sys/types.h>
2 #include <sys/wait.h>
3 pid_t waitpid(pid_t pid, int *status, int options);

```

- pid == -1: aspetta qualsiasi figlio (come wait).
- pid > 0: aspetta il figlio con quel PID.
- options: WNOHANG (non blocca), WUNTRACED (figlio stoppato), WCONTINUED (figlio ripreso).

Esempio di wait con 3 figli che dormono (p. 142):

```

1 for (int i = 1 ; i <= 3 ; i++) {
2     // Fork e ignora i fallimenti di fork.
3     if (fork() == 0) {
4         printf("Child %d sleeps %d seconds...\n", getpid(), i);
5         // Sospende la chiamata del processo per i secondi
6         sleep(i);
7         _exit(0);
8     }
9 }
10
11 pid_t child;

```

```

12 while ((child = wait(NULL)) != -1) {
13     printf("wait() returned child %d\n", child);
14 }
15
16 if (errno != ECHILD) {
17     printf("(wait) An unexpected error...\n");
18 }

```

Output:

```

child 75 sleeps 1 seconds
child 76 sleeps 2 seconds
child 77 sleeps 3 seconds
wait() returned child 75
wait() returned child 76
wait() returned child 77

```

Analisi dello status (pp. 143-144):

Si usano macro per interpretare il valore di `status`:

1. Terminato normalmente:

- `WIFEXITED(status)` è true.
- `WEXITSTATUS(status)` restituisce il codice di uscita.

```

1 waitpid(-1, &status, WUNTRACED | WCONTINUED);
2
3 if (WIFEXITED(status)) {
4     printf("Child exited, status=%d\n", WEXITSTATUS(status));
5 }

```

2. Ucciso da un segnale:

- `WIFSIGNALED(status)` è true.
- `WTERMSIG(status)` restituisce il numero del segnale.

```

1 waitpid(-1, &status, WUNTRACED | WCONTINUED);
2
3 if (WIFSIGNALED(status)) {
4     printf("child killed by signal %d (%s)",
5           WTERMSIG(status), strsignal(WTERMSIG(status)));
6 }

```

3. Stoppato da un segnale:

- `WIFSTOPPED(status)` è true.
- `WSTOPSIG(status)` restituisce il segnale.

```

1 waitpid(-1, &status, WUNTRACED | WCONTINUED);
2 if (WIFSTOPPED(status)) {
3     printf("child stopped by signal %d (%s)\n",
4           WSTOPSIG(status), strsignal(WSTOPSIG(status)));

```

```
5 }
```

4. Ripreso da SIGCONT:

- WIFCONTINUED(status) è true.

15.3.3 8.4 Esecuzione di un Programma: La Famiglia exec (p. 145)

Le funzioni `exec` sostituiscono completamente l'immagine del processo corrente con un nuovo programma. **Non viene creato un nuovo processo** (il PID rimane lo stesso). La funzione base è `execve`.

`execve` (p. 146):

```
1 #include <unistd.h>
2
3 int execve(const char *filename, char *const argv[], char *const envp[]);
```

- `filename`: Path del nuovo eseguibile.
- `argv`: Array di argomenti terminato da NULL.
- `envp`: Array di ambiente terminato da NULL.
- In caso di successo, **non ritorna**. In caso di errore, ritorna **-1**.

Le sei funzioni `exec` (p. 147):

Funzione	Argomenti	Ricerca PATH	Ambiente
<code>execl</code>	Lista	No (path)	Env corrente
<code>execlp</code>	Lista	Sì (file)	Env corrente
<code>execle</code>	Lista	No	Vettore (<code>envp</code>)
<code>execv</code>	Vettore	No	Env corrente
<code>execvp</code>	Vettore	Sì	Env corrente
<code>execve</code>	Vettore	No	Vettore (<code>envp</code>)

(Si veda figura 8.2 a p. 148 per le relazioni)

Esempio fondamentale: `example.c` e `hello.c` (p. 149)

Programma `example.c` (Padre):

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 int main(int argc, char *argv[]) {
5     printf("PID of example.c = %d\n", getpid());
6     char *args[] = {"Hello", "C", "Programming", NULL};
7     execv("./hello", args);
8     printf("Back to example.c");
9     return 0;
10 }
```

Programma `hello.c` (Figlio):

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 int main(int argc, char *argv[]) {
5     printf("We are in hello.c\n");
6     printf("PID of hello.c = %d\n", getpid());
7     return 0;
8 }

```

Compilazione:

```

gcc -o example example.c
gcc -o hello hello.c

```

Esecuzione:

```

PID of example.c = 4733
We are in Hello.c
PID of hello.c = 4733

```

(Il PID è identico perché `execv` ha sostituito `example` con `hello`).

Esempi specifici di `exec` con `printenv` (pp. 149-156): 1. `execl` (p. 149)

```

1 #include <unistd.h>
2 #include <stdio.h>
3 int main (int argc, char *argv[]) {
4     execl("/usr/bin/printenv", "printenv", "HOME", (char *)NULL);
5     perror("Execl");
6 }

```

Output: /home/user (stampa l'ambiente del processo che lo lancia).

2. `execlp` (p. 150)

```

1 #include <unistd.h>
2 #include <stdio.h>
3 int main (int argc, char *argv[]) {
4     execlp("printenv", "printenv", "HOME", (char *)NULL);
5     perror("Execlp");
6 }

```

Output: /home/user (cerca `printenv` nel PATH).

3. `execle` (p. 150)

```

1 #include <unistd.h>
2 #include <stdio.h>
3 int main (int argc, char *argv[]) {
4     char *env[] = {"HOME=/home/pippo", (char *)NULL};
5     execle("/usr/bin/printenv", "printenv", "HOME", (char *)NULL, env);
6     perror("Execle");
7 }

```

Output: /home/pippo (forza l'ambiente passato).

4. `execv` (p. 151)

```
1 #include <unistd.h>
2 #include <stdio.h>
3 int main (int argc, char *argv[]) {
4     char *arg[] = {"printenv", "HOME", (char *)NULL};
5     execv("/usr/bin/printenv", arg);
6     perror("Execv");
7 }
```

Output: /home/user.

5. `execvp` (p. 151)

```
1 #include <unistd.h>
2 #include <stdio.h>
3 int main (int argc, char *argv[]) {
4     char *arg[] = {"printenv", "HOME", (char *)NULL};
5     execvp("printenv", arg);
6     perror("Execvp");
7 }
```

Output: /home/user.

6. `execve` (p. 151)

```
1 #include <unistd.h>
2 #include <stdio.h>
3 int main (int argc, char *argv[]) {
4     char *arg[] = {"printenv", "HOME", (char *)NULL};
5     char *env[] = {"HOME=/home/pippo", (char *)NULL};
6     execve("/usr/bin/printenv", arg, env);
7     perror("Execve");
8 }
```

Output: /home/pippo.

Osservazioni finali sulle `exec`:

- Gli argomenti e i vettori devono sempre terminare con `NULL`.
- Il primo elemento di `argv` è convenzionalmente il nome del programma.
- In caso di successo, **non si torna mai indietro** (le istruzioni successive alla `exec` non vengono eseguite).